```
// ========= File: src/game_layout.h ==========
// src/game_layout.h
#ifndef GAME_LAYOUT_H
#define GAME_LAYOUT_H

#include "splashkit.h"
const int TEXT_CHAR_WIDTH = 8;

// All screen positions and sizes.
const int WINDOW_WIDTH = 960;
const int WINDOW_HEIGHT = 700;

// The 4 x 4 board.
const int BOARD_X = 54;
const int BOARD_Y = 160;
const int CELL_SIZE = 112;
const int CELL_GAP = 10;
const int BOARD_SIZE = 4 * CELL_SIZE + 3 * CELL_GAP;

// Food pictures.
const int FOOD_IMAGE_SIZE = 96;
const int FOOD_MARGIN = (CELL_SIZE - FOOD_IMAGE_SIZE) / 2;

// The three generator buttons sit directly under the board.
const int GENERATOR_Y = BOARD_Y + BOARD_SIZE + 4;
const int GENERATOR_WIDTH = 150;
const int GENERATOR_HEIGHT = 54;
const int GENERATOR_GAP = 14;
const int GENERATOR_ICON_SIZE = 46;
const int DOUGH_BUTTON_X = BOARD_X;
const int TART_BUTTON_X = BOARD_X + GENERATOR_WIDTH + GENERATOR_GAP;
const int CAKE_BUTTON_X = BOARD_X + 2 * (GENERATOR_WIDTH + GENERATOR_GAP);

// Back button in the top-right corner of the play screen.
const int BACK_X = 825;
const int BACK_Y = 14;
const int BACK_WIDTH = 110;
const int BACK_HEIGHT = 36;

// Pause control in the play-screen header.
const int PAUSE_BUTTON_X = 825;
const int PAUSE_BUTTON_Y = 60;
const int PAUSE_BUTTON_WIDTH = 110;
const int PAUSE_BUTTON_HEIGHT = 36;

// Right-hand column of the play screen: customer picture, order, hearts, tips.
const int RIGHT_X = 552;
const int RIGHT_WIDTH = 378;
const int CUSTOMER_BOX_Y = 150;
const int CUSTOMER_BOX_WIDTH = 146;
const int CUSTOMER_BOX_HEIGHT = 144;
const int ORDER_BOX_X = RIGHT_X + CUSTOMER_BOX_WIDTH + 8;
const int ORDER_BOX_WIDTH = RIGHT_X + RIGHT_WIDTH - ORDER_BOX_X;

// The status panel under the order.
```

```
const int STATUS_BOX_Y = 310;
const int STATUS_BOX_HEIGHT = 135;

// Settings button (top-right of the mode-selection screen) and the settings panel.
const int SETTINGS_BUTTON_X = 825;
const int SETTINGS_BUTTON_Y = 14;
const int SETTINGS_BUTTON_WIDTH = 110;
const int SETTINGS_BUTTON_HEIGHT = 36;
const int SETTINGS_BOX_X = 280;
const int SETTINGS_BOX_Y = 170;
const int SETTINGS_BOX_WIDTH = 400;
const int SETTINGS_BOX_HEIGHT = 310;
const int SETTINGS_TOGGLE_X = SETTINGS_BOX_X + 270;      // the ON/OFF buttons
const int SETTINGS_TOGGLE_WIDTH = 100;
const int SETTINGS_TOGGLE_HEIGHT = 40;
const int SETTINGS_MUSIC_Y = SETTINGS_BOX_Y + 80;
const int SETTINGS_EFFECTS_Y = SETTINGS_BOX_Y + 140;
const int SETTINGS_CLOSE_X = SETTINGS_BOX_X + 120;
const int SETTINGS_CLOSE_Y = SETTINGS_BOX_Y + 240;
const int SETTINGS_CLOSE_WIDTH = 160;
const int SETTINGS_CLOSE_HEIGHT = 44;

// Name plate under the story character.
const int STORY_NAME_X = 52;
const int STORY_NAME_Y = 424;
const int STORY_NAME_WIDTH = 223;
const int STORY_NAME_HEIGHT = 34;

// The four mode cards on the mode-selection screen.
const int MODE_CARD_WIDTH = 310;
const int MODE_CARD_HEIGHT = 137;
const int MODE_LEFT_X = 150;
const int MODE_RIGHT_X = 500;
const int MODE_TOP_Y = 165;
const int MODE_BOTTOM_Y = 335;

// The panel behind the text on the win and game-over screens.
const int END_BOX_X = 160;
const int END_BOX_Y = 200;
const int END_BOX_WIDTH = 640;
const int END_BOX_HEIGHT = 250;

// Buttons shown after a round ends.
const int END_REPLAY_BUTTON_X = 300;
const int END_MENU_BUTTON_X = 490;
const int END_BUTTON_Y = END_BOX_Y + 190;
const int END_BUTTON_WIDTH = 170;
const int END_BUTTON_HEIGHT = 44;
const int PAUSE_PANEL_X = 280;
const int PAUSE_PANEL_Y = 265;
const int PAUSE_PANEL_WIDTH = 400;
const int PAUSE_PANEL_HEIGHT = 140;


#endif

```

```
// ========= File: src/utilities.h ==========
// src/utilities.h
#ifndef UTILITIES_H
#define UTILITIES_H

#include "splashkit.h"

// Small helper functions shared by the game and the checks.

// Turn a whole number into text for drawing on screen, e.g. 42 -> "42".
string number_text(int number);

// True when min <= value <= max (the idea behind read_integer_range without any typing).
bool is_in_range(int value, int min, int max);

// True when the point (x, y) is inside the rectangle that starts at (rect_x, rect_y) and is width
pixels wide and height pixels high.
bool point_in_rectangle(double x, double y, int rect_x, int rect_y, int width, int height);

// Estimated width in pixels of a line of text.
int estimated_text_width(string text);

// The x position that centres a line of text inside a box.
int centered_text_x(string text, int box_x, int box_width);

#endif

// ========= File: src/utilities.cpp ==========
// src/utilities.cpp
#include "splashkit.h"
#include "utilities.h"
#include "game_layout.h"

string number_text(int number)
{
    return to_string(number);
}

bool is_in_range(int value, int min, int max)
{
    return value >= min && value <= max;
}

bool point_in_rectangle(double x, double y, int rect_x, int rect_y, int width, int height)
{
    return x >= rect_x && x < rect_x + width && y >= rect_y && y < rect_y + height;
}

int estimated_text_width(string text)
{
    return (int)text.length() * TEXT_CHAR_WIDTH;
}

int centered_text_x(string text, int box_x, int box_width)
{
    return box_x + (box_width - estimated_text_width(text)) / 2;
```

```
}

// ========= File: src/game_state.h ==========
// src/game_state.h
#ifndef GAME_STATE_H
#define GAME_STATE_H

#include "splashkit.h"

const int GRID_SIZE = 4;
// The food atlas has four images per category, using tiers 0 through 3.
const int MAX_ITEM_TIER = 3;
const int ORDER_COUNT = 3;
const int CLASSIC_ORDER_COUNT = 5;
const int MAX_ORDER_COUNT = CLASSIC_ORDER_COUNT;

// Each food chain has its own category.
enum ItemCategory
{
    NO_ITEM,
    DOUGH,
    TART,
    CAKE
};

// Screens and game modes.
enum GameMode
{
    BEGINNER_MODE,
    CLASSIC_MODE,
    RUSH_MODE,
    RECIPE_MODE
};

int order_count_for_mode(GameMode mode);

enum ScreenState
{
    MODE_MENU,
    STORY_SCREEN,
    PLAYING_SCREEN,
    WON_SCREEN,
    GAME_OVER_SCREEN
};

// A board square stores both the food category and its tier.
struct BoardItem
{
    ItemCategory category;
    int tier;
};

struct OrderSpec
{
    ItemCategory category;
    int tier;
```

```
    int quantity;
    ItemCategory secondaryCategory;
    int secondaryTier;
    int secondaryQuantity;
    int moveLimit;
};

struct GameState
{
    // A square with category NO_ITEM is empty.
    BoardItem board[GRID_SIZE][GRID_SIZE];
    int selectedRow, selectedColumn;
    int score, moves, orderIndex, deliveredCount, satisfaction;
    int recipeFirstDelivered, recipeSecondDelivered, movesLeft;
    OrderSpec orders[MAX_ORDER_COUNT];
    unsigned int orderStartedAt;
    string lastAction;

    GameMode mode;
    ScreenState screen;

    int storyLine;
    bool lostByTimer, lostByMoveLimit, lowTimeAlertPlayed;

    // True while "Order complete" is showing and the customer is leaving.
    // The board is locked and the Rush Hour timer is paused until it ends.
    bool orderJustCompleted;
    bool paused;
    unsigned int pausedAt;
    unsigned int orderCompletedAt;
    bool servedFoodRemoved;   // true once the served food has been taken off the board
    bool recipeFoodPending[GRID_SIZE][GRID_SIZE];
    unsigned int recipeFoodDeliveredAt[GRID_SIZE][GRID_SIZE];

    // Player settings. Starting a new round does NOT reset these, so they
    // stay the same until the player changes them.
    bool musicOn;
    bool soundEffectsOn;
    bool settingsOpen;        // true while the settings panel is showing
    bool newHighScore;        // true when this round sets a new local record
    bool resultRecorded;      // prevents saving the same round more than once
};

// Program start: reset everything and show the story first.
void start_new_game(GameState &game);
// Reset everything and show the four-mode selection screen.
void show_mode_menu(GameState &game);
// Start the chosen mode's gameplay immediately.
void choose_mode(GameState &game, GameMode mode, unsigned int now);
ItemCategory current_order_category(const GameState &game);
ItemCategory current_order_secondary_category(const GameState &game);
bool generate_item(GameState &game, ItemCategory category);

// Settings panel on the mode-selection screen. Music and sound effects
// are two separate switches.
void open_settings(GameState &game);
```

```
void close_settings(GameState &game);
void toggle_music(GameState &game);
void toggle_sound_effects(GameState &game);

#endif

// ========= File: src/game_state.cpp ==========
// src/game_state.cpp
#include "splashkit.h"
#include "game_state.h"
#include "sound_manager.h"

int order_count_for_mode(GameMode mode)
{
    return (mode == CLASSIC_MODE) ? CLASSIC_ORDER_COUNT : ORDER_COUNT;
}

// Generate one shuffled pass through all three food categories.
static void shuffled_categories(ItemCategory categories[3])
{
    categories[0] = DOUGH;
    categories[1] = TART;
    categories[2] = CAKE;
    for (int i = 2; i > 0; i--)
    {
        int swapIndex = rnd(i + 1);   // rnd(n) gives 0 to n - 1, so this is 0 to i
        ItemCategory temp = categories[i];
        categories[i] = categories[swapIndex];
        categories[swapIndex] = temp;
    }
}

// Store the orders once when a mode starts so they stay stable during play.
static void setup_orders(GameState &game, GameMode mode)
{
    ItemCategory categories[3];
    if (mode == BEGINNER_MODE)
    {
        categories[0] = DOUGH;
        categories[1] = TART;
        categories[2] = CAKE;
    }
    else
    {
        shuffled_categories(categories);
    }

    for (int i = 0; i < order_count_for_mode(mode); i++)
    {
        OrderSpec &order = game.orders[i];
        order.category = (mode == BEGINNER_MODE) ? (ItemCategory)(DOUGH + i) : categories[i %
ORDER_COUNT];
        order.tier = (mode == BEGINNER_MODE) ? ((i == 2) ? 3 : 2) : 1 + rnd(2);
        order.quantity = 1;
        order.secondaryCategory = NO_ITEM;
        order.secondaryTier = -1;
```

```
        order.secondaryQuantity = 0;
        order.moveLimit = 12 + i * 4;

        if (mode == CLASSIC_MODE)
            order.quantity = 1 + rnd(2);   // 1 or 2 items
        else if (mode == RECIPE_MODE)
        {
            order.quantity = 1 + rnd(2);   // 1 or 2 items of the first food
            order.secondaryCategory = categories[(i + 1) % ORDER_COUNT];
            order.secondaryTier = 1 + rnd(2);
            order.secondaryQuantity = 1;
        }
    }
}

// Empty every square of the board.
static void setup_board(GameState &game)
{
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int column = 0; column < GRID_SIZE; column++)
        {
            game.board[row][column].category = NO_ITEM;
            game.board[row][column].tier = 0;
            game.recipeFoodPending[row][column] = false;
            game.recipeFoodDeliveredAt[row][column] = 0;
        }
    }
}

// Reset every value that changes during a round.
static void reset_round(GameState &game)
{
    game.mode = BEGINNER_MODE;
    game.score = 0;
    game.moves = 0;
    game.orderIndex = 0;
    game.deliveredCount = 0;
    game.satisfaction = 3;
    game.recipeFirstDelivered = 0;
    game.recipeSecondDelivered = 0;
    game.movesLeft = 0;
    game.storyLine = 0;
    game.orderStartedAt = 0;
    game.lostByTimer = false;
    game.lostByMoveLimit = false;
    game.lowTimeAlertPlayed = false;
    game.orderJustCompleted = false;
    game.paused = false;
    game.pausedAt = 0;
    game.orderCompletedAt = 0;
    game.servedFoodRemoved = false;
    game.lastAction = "";
    game.settingsOpen = false;   // the panel is closed, but music/effects settings are kept
    game.newHighScore = false;
    game.resultRecorded = false;
```

```
    game.selectedRow = -1;
    game.selectedColumn = -1;
    setup_orders(game, BEGINNER_MODE);
    setup_board(game);
}

// Called once when the program starts: the story scene comes first.
void start_new_game(GameState &game)
{
    reset_round(game);
    game.musicOn = true;           // both switches start ON; reset_round never changes them
    game.soundEffectsOn = true;
    game.screen = STORY_SCREEN;
}

// Used by the Back button, the R key and the end screens.
// The story is not shown again.
void show_mode_menu(GameState &game)
{
    if (game.paused)
        apply_music_setting(game);
    reset_round(game);
    game.screen = MODE_MENU;
}

// Start the selected mode and go straight to gameplay.
void choose_mode(GameState &game, GameMode mode, unsigned int now)
{
    reset_round(game);
    game.mode = mode;
    setup_orders(game, mode);
    game.movesLeft = (mode == RECIPE_MODE) ? game.orders[0].moveLimit : 0;
    game.orderStartedAt = now;
    game.screen = PLAYING_SCREEN;
}

void open_settings(GameState &game)
{
    game.settingsOpen = true;
}

void close_settings(GameState &game)
{
    game.settingsOpen = false;
}

// Each switch only changes its own setting.
void toggle_music(GameState &game)
{
    game.musicOn = !game.musicOn;
}

void toggle_sound_effects(GameState &game)
{
    game.soundEffectsOn = !game.soundEffectsOn;
}
```

```

// ========= File: src/game_logic.h ==========
// src/game_logic.h
#ifndef GAME_LOGIC_H
#define GAME_LOGIC_H

#include "splashkit.h"
#include "game_state.h"

// Rush Hour gives the player 30 seconds to complete each order.
const unsigned int ORDER_TIME_LIMIT = 30000;

// After an order is finished (times in milliseconds):
// 0 - 1000: "Order complete" message, customer still shown
// 1000 - 1600: customer has left, waiting for the next one
// 1600: next customer arrives with a new order
const unsigned int ORDER_COMPLETE_MESSAGE_TIME = 1000;

// Rush Hour: every customer has 3 hearts. One heart is lost every 10 seconds
// of waiting, and the customer leaves (game over) after 30 seconds.
const int MAX_HEARTS = 3;
const unsigned int CUSTOMER_HEART_TIME = 10000;

const unsigned int CUSTOMER_TRANSITION_TIME = 1600;

// Change screens, update timers and handle cake movement and merging.
void advance_screen(GameState &game, unsigned int now);
void update_game(GameState &game, unsigned int now);
void toggle_pause(GameState &game, unsigned int now);
void select_or_move_cake(GameState &game, int row, int column, unsigned int now);

// Return the current order requirements for the user interface.
int current_order_tier(const GameState &game);
int current_order_quantity(const GameState &game);
int current_order_secondary_tier(const GameState &game);
int current_order_secondary_quantity(const GameState &game);
unsigned int current_order_time_left(const GameState &game, unsigned int now);

// After an order is finished the customer leaves, then the next one arrives.
// True once the "Order complete" message is over and the customer has gone.
// Recipe Challenge: how many moves the current order allows in total.
int current_move_limit(const GameState &game);

bool customer_has_left(const GameState &game, unsigned int now);

#endif

// ========= File: src/game_logic.cpp ==========
// src/game_logic.cpp
#include "splashkit.h"
#include "game_state.h"
#include "game_logic.h"
#include "sound_manager.h"
#include "utilities.h"

static void clear_selection(GameState &game)
```

```
{
    game.selectedRow = -1;
    game.selectedColumn = -1;
}

// Story: move through the dialogue, then show the four modes.
// Win or game-over screen: return to the four modes.
void advance_screen(GameState &game, unsigned int now)
{
    (void)now; // kept so existing calls stay the same

    if (game.screen == STORY_SCREEN)
    {
        if (game.storyLine < 2)
            game.storyLine++;
        else
            show_mode_menu(game);
    }
    else if (game.screen == WON_SCREEN || game.screen == GAME_OVER_SCREEN)
    {
        show_mode_menu(game);
    }
}

ItemCategory current_order_category(const GameState &game)
{
    if (game.orderIndex >= order_count_for_mode(game.mode)) return NO_ITEM;

    return game.orders[game.orderIndex].category;
}

int current_order_tier(const GameState &game)
{
    if (game.orderIndex >= order_count_for_mode(game.mode)) return -1;

    return game.orders[game.orderIndex].tier;
}

int current_order_quantity(const GameState &game)
{
    if (game.orderIndex >= order_count_for_mode(game.mode)) return 0;

    return game.orders[game.orderIndex].quantity;
}

ItemCategory current_order_secondary_category(const GameState &game)
{
    if (game.mode != RECIPE_MODE || game.orderIndex >= order_count_for_mode(game.mode))
        return NO_ITEM;
    return game.orders[game.orderIndex].secondaryCategory;
}

int current_order_secondary_tier(const GameState &game)
{
    if (game.mode != RECIPE_MODE || game.orderIndex >= order_count_for_mode(game.mode))
        return -1;
```

```
    return game.orders[game.orderIndex].secondaryTier;
}

int current_order_secondary_quantity(const GameState &game)
{
    if (game.mode != RECIPE_MODE || game.orderIndex >= order_count_for_mode(game.mode))
        return 0;
    return game.orders[game.orderIndex].secondaryQuantity;
}

unsigned int current_order_time_left(const GameState &game, unsigned int now)
{
    if (game.mode != RUSH_MODE || game.screen != PLAYING_SCREEN)
        return ORDER_TIME_LIMIT;
    // The timer is paused while the customer is leaving.
    unsigned int clock = game.orderJustCompleted ? game.orderCompletedAt : (game.paused ? game.pausedAt
: now);
    unsigned int elapsed = clock - game.orderStartedAt;
    return (elapsed >= ORDER_TIME_LIMIT) ? 0 : ORDER_TIME_LIMIT - elapsed;
}

int current_move_limit(const GameState &game)
{
    if (game.mode != RECIPE_MODE || game.orderIndex >= order_count_for_mode(game.mode)) return 0;
    return game.orders[game.orderIndex].moveLimit;
}

// Rush Hour: hearts the current customer has left, based on how long they have waited.
static int hearts_left(const GameState &game, unsigned int now)
{
    // The clock is paused while the customer is leaving.
    unsigned int clock = game.orderJustCompleted ? game.orderCompletedAt : (game.paused ? game.pausedAt
: now);
    int heartsLost = (int)((clock - game.orderStartedAt) / CUSTOMER_HEART_TIME);
    if (heartsLost >= MAX_HEARTS) return 0;
    return MAX_HEARTS - heartsLost;
}

bool customer_has_left(const GameState &game, unsigned int now)
{
    unsigned int clock = game.paused ? game.pausedAt : now;
    return game.orderJustCompleted && clock - game.orderCompletedAt >= ORDER_COMPLETE_MESSAGE_TIME;
}

static bool can_make_a_move(const GameState &game)
{
    // An empty square means the player can still generate or move an item.
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int col = 0; col < GRID_SIZE; col++)
        {
            if (game.board[row][col].category == NO_ITEM)
                return true;
        }
    }

```

```
    // Matching categories and tiers mean that a merge is possible.
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int col = 0; col < GRID_SIZE; col++)
        {
            BoardItem first = game.board[row][col];
            if (first.category == NO_ITEM || first.tier >= MAX_ITEM_TIER)
                continue;
            for (int otherRow = 0; otherRow < GRID_SIZE; otherRow++)
            {
                for (int otherCol = 0; otherCol < GRID_SIZE; otherCol++)
                {
                    if (row == otherRow && col == otherCol)
                        continue;
                    BoardItem second = game.board[otherRow][otherCol];
                    if (first.category == second.category && first.tier == second.tier)
                        return true;
                }
            }
        }
    }
    return false;
}

static bool recipe_food_is_pending(const GameState &game, int row, int column)
{
    return game.recipeFoodPending[row][column];
}

static void update_recipe_food_collection(GameState &game, unsigned int now)
{
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int column = 0; column < GRID_SIZE; column++)
        {
            if (game.recipeFoodPending[row][column] &&
                now - game.recipeFoodDeliveredAt[row][column] >= ORDER_COMPLETE_MESSAGE_TIME)
            {
                game.board[row][column].category = NO_ITEM;
                game.board[row][column].tier = 0;
                game.recipeFoodPending[row][column] = false;
            }
        }
    }
}
// Take up to "count" foods of one kind off the board (rows first, then columns).
static void remove_foods(GameState &game, ItemCategory category, int tier, int count)
{
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int col = 0; col < GRID_SIZE; col++)
        {
            if (count > 0 && game.board[row][col].category == category && game.board[row][col].tier ==
tier)
            {
                game.board[row][col].category = NO_ITEM;
```

```
                game.board[row][col].tier = 0;
                count--;
            }
        }
    }
}

// The foods that were counted for the current order are taken away
static void remove_served_food(GameState &game)
{
    if (game.orderIndex >= order_count_for_mode(game.mode)) return;

    // Recipe items are removed as soon as each exact requirement is delivered.
    // Other modes still remove their completed items when the customer leaves.
    if (game.mode != RECIPE_MODE)
        remove_foods(game, current_order_category(game), current_order_tier(game),
game.deliveredCount);
}

// Generates the requested food type in a randomly chosen empty square.
// Returns false if the board has no empty square.
bool generate_item(GameState &game, ItemCategory category)
{
    if (game.screen != PLAYING_SCREEN || category == NO_ITEM)
        return false;
    int emptyCount = 0;
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int col = 0; col < GRID_SIZE; col++)
        {
            if (game.board[row][col].category == NO_ITEM)
                emptyCount++;
        }
    }

    if (emptyCount == 0)
        return false;

    // Pick one of the empty squares at random.
    int chosenEmpty = rnd(emptyCount);   // rnd(n) gives 0 to n - 1
    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int col = 0; col < GRID_SIZE; col++)
        {
            if (game.board[row][col].category == NO_ITEM)
            {
                if (chosenEmpty == 0)
                {
                    game.board[row][col].category = category;
                    game.board[row][col].tier = 0;
                    game.lastAction = "";
                    play_drop_sound(game);
                    return true;
                }
                chosenEmpty--;
            }
```

```
        }
    }
    return false;
}

static bool order_is_complete(const GameState &game)
{
    if (game.mode == RECIPE_MODE)
    {
        return game.recipeFirstDelivered >= current_order_quantity(game) && game.recipeSecondDelivered >= current_order_secondary_quantity(game);
    }
    return game.deliveredCount >= current_order_quantity(game);
}

// The next customer arrives with the next order.
static void finish_order_transition(GameState &game, unsigned int now)
{
    game.orderJustCompleted = false;
    game.servedFoodRemoved = false;
    game.orderIndex++;
    game.deliveredCount = 0;
    game.recipeFirstDelivered = 0;
    game.recipeSecondDelivered = 0;
    game.lastAction = "";
    game.orderStartedAt = now;   // Rush Hour: the new order gets a full timer
    game.lowTimeAlertPlayed = false;
    game.satisfaction = MAX_HEARTS;   // every new customer starts with full hearts

    if (game.mode == RECIPE_MODE)
        game.movesLeft = current_move_limit(game);

    play_question_sound(game);
}

static void complete_order(GameState &game, unsigned int now)
{
    if (!order_is_complete(game))
        return;
    play_order_sound(game);
    game.score += 100 * (game.orderIndex + 1);

    if (game.mode == RUSH_MODE)
        game.score += (int)(current_order_time_left(game, now) / 1000) * 2;

    // The last order goes straight to the win screen.
    if (game.orderIndex + 1 >= order_count_for_mode(game.mode))
    {
        game.orderIndex++;
        game.screen = WON_SCREEN;
        return;
    }

    // Keep the hearts this customer had when they were served.
    if (game.mode == RUSH_MODE)
        game.satisfaction = hearts_left(game, now);
```

```
777
778     // Otherwise show "Order complete" and let the customer leave first.
779     // The order number changes when the next customer arrives.
780     game.orderJustCompleted = true;
781     game.servedFoodRemoved = false;
782     game.orderCompletedAt = now;
783 }
784
785 static void record_item_for_order(GameState &game, ItemCategory category, int tier,
786                                   int row, int column, unsigned int now)
787 {
788     // Only an exact category and tier advances an order. Recipe deliveries are
789     // consumed immediately so a completed ingredient cannot be merged again.
790     bool counted = false;
791     bool requestedCategory = false;
792     bool requirementAlreadyMet = false;
793     int requestedTier = -1;
794
795     if (game.mode == RECIPE_MODE)
796     {
797         if (category == current_order_category(game))
798         {
799             requestedCategory = true;
800             requestedTier = current_order_tier(game);
801             requirementAlreadyMet = game.recipeFirstDelivered >= current_order_quantity(game);
802             if (tier == requestedTier && !requirementAlreadyMet)
803             {
804                 game.recipeFirstDelivered++;
805                 counted = true;
806             }
807         }
808         else if (category == current_order_secondary_category(game))
809         {
810             requestedCategory = true;
811             requestedTier = current_order_secondary_tier(game);
812             requirementAlreadyMet = game.recipeSecondDelivered >= current_order_second-
ary_quantity(game);
813             if (tier == requestedTier && !requirementAlreadyMet)
814             {
815                 game.recipeSecondDelivered++;
816                 counted = true;
817             }
818         }
819     }
820     else if (category == current_order_category(game))
821     {
822         requestedCategory = true;
823         requestedTier = current_order_tier(game);
824         requirementAlreadyMet = game.deliveredCount >= current_order_quantity(game);
825         if (tier == requestedTier && !requirementAlreadyMet)
826         {
827             game.deliveredCount++;
828             counted = true;
829         }
830     }
831
```

```
    if (counted)
    {
        game.lastAction = "Counted: level " + number_text(tier + 1) + ".";
        if (game.mode == RECIPE_MODE)
        {
            game.recipeFoodPending[row][column] = true;
            game.recipeFoodDeliveredAt[row][column] = now;
        }
    }
    else if (requirementAlreadyMet)
        game.lastAction = "That recipe item is already complete.";
    else if (requestedCategory)
        game.lastAction = "Not counted: needs level " + number_text(requestedTier + 1) + ".";
    else
        game.lastAction = "This food is not in the order.";

    complete_order(game, now);
}

static void use_move(GameState &game)
{
    game.moves++;
    if (game.mode == RECIPE_MODE)
        game.movesLeft--;
}

// Recipe Challenge: running out of moves before the recipe is finished ends the game.
static void check_recipe_limit(GameState &game)
{
    if (game.mode == RECIPE_MODE &&
        game.screen == PLAYING_SCREEN &&
        !game.orderJustCompleted &&
        game.movesLeft <= 0 &&
        game.orderIndex < order_count_for_mode(game.mode))
    {
        play_timeout_sound(game);
        game.screen = GAME_OVER_SCREEN;
        game.lostByMoveLimit = true;
    }
}

void select_or_move_cake(GameState &game, int row, int column, unsigned int now)
{
    // The board is locked while the customer is leaving.
    if (game.screen != PLAYING_SCREEN ||
        game.orderJustCompleted ||
        !is_in_range(row, 0, GRID_SIZE - 1) ||
        !is_in_range(column, 0, GRID_SIZE - 1))
        return;

    if (game.mode == RECIPE_MODE && recipe_food_is_pending(game, row, column))
    {
        clear_selection(game);
        game.lastAction = "This item is being served.";
        return;
    }
```

```
888
889     BoardItem target = game.board[row][column];
890     // First click selects an occupied square.
891     if (game.selectedRow == -1)
892     {
893         if (target.category != NO_ITEM)
894         {
895             game.lastAction = "";
896             game.selectedRow = row;
897             game.selectedColumn = column;
898         }
899         return;
900     }
901
902     int fromRow = game.selectedRow;
903     int fromColumn = game.selectedColumn;
904
905     if (fromRow == row && fromColumn == column)
906     {
907         game.lastAction = "";
908         clear_selection(game);
909         return;
910     }
911     BoardItem moving = game.board[fromRow][fromColumn];
912
913     // Move an item into an empty square.
914     if (target.category == NO_ITEM)
915     {
916         game.lastAction = "";
917         game.board[row][column] = moving;
918         game.board[fromRow][fromColumn].category = NO_ITEM;
919         game.board[fromRow][fromColumn].tier = 0;
920
921         play_drop_sound(game);
922         use_move(game);
923         check_recipe_limit(game);
924         clear_selection(game);
925         return;
926     }
927
928     // Merge only when both the category and tier match.
929     if (target.category == moving.category && target.tier == moving.tier && moving.tier <
    MAX_ITEM_TIER)
930     {
931         int mergedTier = moving.tier + 1;
932         game.board[row][column].tier = mergedTier;
933         game.board[fromRow][fromColumn].category = NO_ITEM;
934         game.board[fromRow][fromColumn].tier = 0;
935
936         play_merge_sound(game);
937         game.score += mergedTier * 10;
938         use_move(game);
939         record_item_for_order(game, target.category, mergedTier, row, column, now);
940         check_recipe_limit(game);
941         clear_selection(game);
942         return;
```

```
    }

    // Clicking a different occupied square selects that square instead.
    game.lastAction = "";
    game.selectedRow = row;
    game.selectedColumn = column;
}

void toggle_pause(GameState &game, unsigned int now)
{
    if (game.screen != PLAYING_SCREEN)
        return;

    if (!game.paused)
    {
        game.paused = true;
        game.pausedAt = now;
        pause_music();
    }
    else
    {
        unsigned int pausedDuration = now - game.pausedAt;
        for (int row = 0; row < GRID_SIZE; row++)
            for (int column = 0; column < GRID_SIZE; column++)
                if (game.recipeFoodPending[row][column])
                    game.recipeFoodDeliveredAt[row][column] += pausedDuration;
        game.orderStartedAt += pausedDuration;
        if (game.orderJustCompleted)
            game.orderCompletedAt += pausedDuration;
        game.paused = false;
        game.pausedAt = 0;
        apply_music_setting(game);
    }
}

// Called every frame: timers, the customer leaving, and the "no moves left" check.
void update_game(GameState &game, unsigned int now)
{
    if (game.screen != PLAYING_SCREEN || game.paused) return;
    if (game.mode == RECIPE_MODE)
        update_recipe_food_collection(game, now);
    // While the customer is leaving nothing else can happen (the timer is paused).
    if (game.orderJustCompleted)
    {
        // When the customer leaves, they take the food they ordered with them.
        if (customer_has_left(game, now) && !game.servedFoodRemoved)
        {
            remove_served_food(game);
            game.servedFoodRemoved = true;
        }

        if (now - game.orderCompletedAt >= CUSTOMER_TRANSITION_TIME)
            finish_order_transition(game, now);

        return;
    }
```

```
999
1000    if (game.mode == RUSH_MODE)
1001    {
1002        // A heart is lost every 10 seconds, with a tick to tell the player.
1003        int hearts = hearts_left(game, now);
1004
1005        if (hearts < game.satisfaction)
1006            play_tick_sound(game);
1007
1008        game.satisfaction = hearts;
1009
1010        // Last 10 seconds: one more warning tick.
1011        if (!game.lowTimeAlertPlayed && current_order_time_left(game, now) <= 10000 &&
    current_order_time_left(game, now) > 0)
1012        {
1013            play_tick_sound(game);
1014            game.lowTimeAlertPlayed = true;
1015        }
1016
1017        // After 30 seconds the customer leaves and the game is over.
1018        if (current_order_time_left(game, now) == 0)
1019        {
1020            play_timeout_sound(game);
1021            game.screen = GAME_OVER_SCREEN;
1022            game.lostByTimer = true;
1023            return;
1024        }
1025    }
1026
1027    if (game.screen == PLAYING_SCREEN && !can_make_a_move(game))
1028        game.screen = GAME_OVER_SCREEN;
1029 }
1030
1031 // ========= File: src/score_manager.h ==========
1032 //src/score_manager.h
1033 #ifndef SCORE_MANAGER_H
1034 #define SCORE_MANAGER_H
1035
1036 #include "game_state.h"
1037
1038 int high_score_for_mode(GameMode mode);
1039 bool record_high_score(GameMode mode, int score);
1040
1041 #endif
1042
1043 // ========= File: src/score_manager.cpp ==========
1044 //src/score_manager.cpp
1045 #include "splashkit.h"
1046 #include "score_manager.h"
1047 #include <fstream>
1048
1049 static const int MODE_COUNT = 3;
1050 static int high_scores[MODE_COUNT] = {0, 0, 0};
1051 static bool scores_loaded = false;
1052
1053 static int mode_index(GameMode mode)
```

```
{
    // Beginner has a fixed learning recipe and does not keep a high mark.
    switch (mode)
    {
        case CLASSIC_MODE: return 0;
        case RUSH_MODE: return 1;
        case RECIPE_MODE: return 2;
        default: return -1;
    }
}

static void load_high_scores()
{
    if (scores_loaded) return;

    std::ifstream input("high_scores.txt");
    for (int i = 0; i < MODE_COUNT; i++)
    {
        int score = 0;
        if (input >> score && score >= 0)
            high_scores[i] = score;
    }
    scores_loaded = true;
}

static void save_high_scores()
{
    std::ofstream output("high_scores.txt", std::ios::trunc);
    if (!output)
    {
        write_line("Could not save high scores. Run the game from its project folder.");
        return;
    }

    for (int i = 0; i < MODE_COUNT; i++)
        output << high_scores[i] << '\n';
}

int high_score_for_mode(GameMode mode)
{
    load_high_scores();
    int index = mode_index(mode);
    return (index < 0) ? 0 : high_scores[index];
}

bool record_high_score(GameMode mode, int score)
{
    load_high_scores();
    int index = mode_index(mode);
    if (index < 0 || score <= high_scores[index]) return false;

    high_scores[index] = score;
    save_high_scores();
    return true;
}

```

```
// ========= File: src/sound_manager.h ==========
// src/sound_manager.h
#ifndef SOUND_MANAGER_H
#define SOUND_MANAGER_H

#include "splashkit.h"
#include "game_state.h"

void load_game_sounds();
void play_story_voice(const GameState &game, int story_line);
void stop_story_voice();
void load_background_music();
void play_background_music();
// Pause or resume the music to match the player's music setting.
void apply_music_setting(const GameState &game);
void play_merge_sound(const GameState &game);
void play_order_sound(const GameState &game);
void play_timeout_sound(const GameState &game);
void play_drop_sound(const GameState &game);
void play_click_sound(const GameState &game);
void play_question_sound(const GameState &game);
void play_tick_sound(const GameState &game);

#endif

// ========= File: src/sound_manager.cpp ==========
// src/sound_manager.cpp
#include "splashkit.h"
#include "sound_manager.h"

void load_game_sounds()
{
    // Load every short effect once before the game loop starts.
    load_sound_effect("merge", "Resources/audio/merge_pop.wav");
    load_sound_effect("order", "Resources/audio/confirmation.ogg");
    load_sound_effect("timeout", "Resources/audio/error.ogg");
    load_sound_effect("drop", "Resources/audio/drop.ogg");
    load_sound_effect("click", "Resources/audio/click.ogg");
    load_sound_effect("question", "Resources/audio/question.ogg");
    load_sound_effect("tick", "Resources/audio/tick.ogg");

    // Story narration for Mia and the festival guest.
    load_sound_effect("story_mia_intro", "Resources/audio/story_mia_intro.mp3");
    load_sound_effect("story_festival_guest", "Resources/audio/story_festival_guest.mp3");
    load_sound_effect("story_mia_invitation", "Resources/audio/story_mia_invitation.mp3");
}

void stop_story_voice()
{
    stop_sound_effect("story_mia_intro");
    stop_sound_effect("story_festival_guest");
    stop_sound_effect("story_mia_invitation");
}

void play_story_voice(const GameState &game, int story_line)
{
```

```
    stop_story_voice();
    if (!game.soundEffectsOn) return;

    if (story_line == 0)
        play_sound_effect("story_mia_intro", 0.85);
    else if (story_line == 1)
        play_sound_effect("story_festival_guest", 0.85);
    else if (story_line == 2)
        play_sound_effect("story_mia_invitation", 0.85);
}

// Play one effect, but only when the player has sound effects switched on.
static void play_effect(const GameState &game, const string &name, double volume)
{
    if (game.soundEffectsOn)
        play_sound_effect(name, volume);
}

// Keep effects slightly below full volume so they do not overpower the game.
void play_merge_sound(const GameState &game)
{
    play_effect(game, "merge", 0.75);
}

void play_order_sound(const GameState &game)
{
    play_effect(game, "order", 0.7);
}

void play_timeout_sound(const GameState &game)
{
    play_effect(game, "timeout", 0.55);
}

void play_drop_sound(const GameState &game)
{
    play_effect(game, "drop", 0.45);
}

void play_click_sound(const GameState &game)
{
    play_effect(game, "click", 0.45);
}

void play_question_sound(const GameState &game)
{
    play_effect(game, "question", 0.45);
}

void play_tick_sound(const GameState &game)
{
    play_effect(game, "tick", 0.35);
}

// Load and play a soft bakery theme throughout the game.
static music background_music;
```

```
void load_background_music()
{
    background_music = load_music("bakery_bgm", "Resources/audio/bakery_bgm.mp3");
}

void play_background_music()
{
    play_music(background_music, -1, 0.20);
}

// The music keeps its place in the song while it is paused.
void apply_music_setting(const GameState &game)
{
    if (game.musicOn)
        resume_music();
    else
        pause_music();
}

// ========= File: src/input.h ==========
// src/input.h
#ifndef INPUT_H
#define INPUT_H

#include "splashkit.h"
#include "game_state.h"

// Route a mouse click to a menu button or a board cell.
void handle_mouse_click(GameState &game, double mouseX, double mouseY, unsigned int now);

// Handle the keyboard controls for the current screen.
void handle_key_presses(GameState &game, unsigned int now);

#endif

// ========= File: src/input.cpp ==========
// src/input.cpp
#include "splashkit.h"
#include "input.h"
#include "game_logic.h"
#include "game_layout.h"
#include "sound_manager.h"
#include "utilities.h"

// Stop the current narration when the player advances, then speak the next subtitle.
static void advance_story_with_voice(GameState &game, unsigned int now)
{
    stop_story_voice();
    advance_screen(game, now);
    if (game.screen == STORY_SCREEN)
        play_story_voice(game, game.storyLine);
}

// Return which generator was clicked, or NO_ITEM if the click missed them all.
static ItemCategory generator_at(double mouseX, double mouseY)
{
```

```
    if (point_in_rectangle(mouseX, mouseY, DOUGH_BUTTON_X, GENERATOR_Y, GENERATOR_WIDTH,
GENERATOR_HEIGHT))
        return DOUGH;

    if (point_in_rectangle(mouseX, mouseY, TART_BUTTON_X, GENERATOR_Y, GENERATOR_WIDTH,
GENERATOR_HEIGHT))
        return TART;

    if (point_in_rectangle(mouseX, mouseY, CAKE_BUTTON_X, GENERATOR_Y, GENERATOR_WIDTH,
GENERATOR_HEIGHT))
        return CAKE;

    return NO_ITEM;
}

// Handle a click inside the settings panel. The two switches work independently,
// and clicks outside the panel do nothing while it is open.
static void handle_settings_click(GameState &game, double mouseX, double mouseY)
{
    if (point_in_rectangle(mouseX, mouseY, SETTINGS_TOGGLE_X, SETTINGS_MUSIC_Y,
            SETTINGS_TOGGLE_WIDTH, SETTINGS_TOGGLE_HEIGHT))
    {
        toggle_music(game);
        apply_music_setting(game);
        play_click_sound(game);
    }
    else if (point_in_rectangle(mouseX, mouseY, SETTINGS_TOGGLE_X, SETTINGS_EFFECTS_Y,
            SETTINGS_TOGGLE_WIDTH, SETTINGS_TOGGLE_HEIGHT))
    {
        toggle_sound_effects(game);
        play_click_sound(game);   // only heard when effects have just been switched ON
    }
    else if (point_in_rectangle(mouseX, mouseY, SETTINGS_CLOSE_X, SETTINGS_CLOSE_Y,
            SETTINGS_CLOSE_WIDTH, SETTINGS_CLOSE_HEIGHT))
    {
        play_click_sound(game);
        close_settings(game);
    }
}

// Handle a click on the four mode cards. Choosing a mode starts gameplay at once.
static void handle_menu_click(GameState &game, double mouseX, double mouseY, unsigned int now)
{
    if (game.settingsOpen)
    {
        handle_settings_click(game, mouseX, mouseY);
        return;
    }

    if (point_in_rectangle(mouseX, mouseY, SETTINGS_BUTTON_X, SETTINGS_BUTTON_Y,
            SETTINGS_BUTTON_WIDTH, SETTINGS_BUTTON_HEIGHT))
    {
        play_click_sound(game);
        open_settings(game);
        return;
    }
```

```
1331
1332        GameMode chosen = BEGINNER_MODE;
1333        bool clickedCard = true;
1334
1335        if (point_in_rectangle(mouseX, mouseY, MODE_LEFT_X, MODE_TOP_Y, MODE_CARD_WIDTH, MODE_CARD_HEIGHT))
1336            chosen = BEGINNER_MODE;
1337        else if (point_in_rectangle(mouseX, mouseY, MODE_RIGHT_X, MODE_TOP_Y, MODE_CARD_WIDTH,
    MODE_CARD_HEIGHT))
1338            chosen = CLASSIC_MODE;
1339        else if (point_in_rectangle(mouseX, mouseY, MODE_LEFT_X, MODE_BOTTOM_Y, MODE_CARD_WIDTH,
    MODE_CARD_HEIGHT))
1340            chosen = RUSH_MODE;
1341        else if (point_in_rectangle(mouseX, mouseY, MODE_RIGHT_X, MODE_BOTTOM_Y, MODE_CARD_WIDTH,
    MODE_CARD_HEIGHT))
1342            chosen = RECIPE_MODE;
1343        else
1344            clickedCard = false;
1345
1346        if (clickedCard)
1347        {
1348            play_click_sound(game);
1349            choose_mode(game, chosen, now);
1350        }
1351    }
1352
1353    // Handle a click on the 4 x 4 board: select, move or merge.
1354    static void handle_board_click(GameState &game, double mouseX, double mouseY, unsigned int now)
1355    {
1356        if (!point_in_rectangle(mouseX, mouseY, BOARD_X, BOARD_Y, BOARD_SIZE, BOARD_SIZE))
1357            return;
1358
1359        int column = (int)(mouseX - BOARD_X) / (CELL_SIZE + CELL_GAP);
1360        int row = (int)(mouseY - BOARD_Y) / (CELL_SIZE + CELL_GAP);
1361        int inCellX = (int)(mouseX - BOARD_X) % (CELL_SIZE + CELL_GAP);
1362        int inCellY = (int)(mouseY - BOARD_Y) % (CELL_SIZE + CELL_GAP);
1363
1364        // Ignore the small gaps between board squares.
1365        if (inCellX < CELL_SIZE && inCellY < CELL_SIZE)
1366        {
1367            select_or_move_cake(game, row, column, now);
1368        }
1369    }
1370
1371    void handle_mouse_click(GameState &game, double mouseX, double mouseY, unsigned int now)
1372    {
1373        if (game.screen == MODE_MENU)
1374        {
1375            handle_menu_click(game, mouseX, mouseY, now);
1376            return;
1377        }
1378
1379        if (game.screen == STORY_SCREEN)
1380        {
1381            play_click_sound(game);
1382            advance_story_with_voice(game, now);
1383            return;
```

```
    }

    if (game.screen == WON_SCREEN || game.screen == GAME_OVER_SCREEN)
    {
        if (point_in_rectangle(mouseX, mouseY, END_REPLAY_BUTTON_X, END_BUTTON_Y,
                END_BUTTON_WIDTH, END_BUTTON_HEIGHT))
        {
            play_click_sound(game);
            choose_mode(game, game.mode, now);
        }
        else if (point_in_rectangle(mouseX, mouseY, END_MENU_BUTTON_X, END_BUTTON_Y,
                END_BUTTON_WIDTH, END_BUTTON_HEIGHT))
        {
            play_click_sound(game);
            show_mode_menu(game);
        }
        else
        {
            play_click_sound(game);
            advance_screen(game, now);
        }
        return;
    }

    if (game.screen != PLAYING_SCREEN)
        return;

    // Pause/Resume remains available while the game is paused.
    if (point_in_rectangle(mouseX, mouseY, PAUSE_BUTTON_X, PAUSE_BUTTON_Y,
            PAUSE_BUTTON_WIDTH, PAUSE_BUTTON_HEIGHT))
    {
        play_click_sound(game);
        toggle_pause(game, now);
        return;
    }

    // The BACK button returns to mode selection without closing the game.
    if (point_in_rectangle(mouseX, mouseY, BACK_X, BACK_Y, BACK_WIDTH, BACK_HEIGHT))
    {
        play_click_sound(game);
        show_mode_menu(game);
        return;
    }

    if (game.paused)
        return;

    // A generator button puts its food in a random empty square straight away.
    ItemCategory generator = generator_at(mouseX, mouseY);

    if (generator != NO_ITEM)
    {
        if (generate_item(game, generator))
            play_click_sound(game);
        else
            play_timeout_sound(game); // the board is full
```

```
        return;
    }

    handle_board_click(game, mouseX, mouseY, now);
}

// Keyboard controls: 1-4 pick a mode, SPACE continues, R goes back to the modes.
void handle_key_presses(GameState &game, unsigned int now)
{
    if (game.screen == MODE_MENU)
    {
        // While the settings panel is open only ESC works.
        if (game.settingsOpen)
        {
            if (key_typed(ESCAPE_KEY))
                close_settings(game);
            return;
        }

        if (key_typed(NUM_1_KEY))
        {
            play_click_sound(game);
            choose_mode(game, BEGINNER_MODE, now);
        }
        else if (key_typed(NUM_2_KEY))
        {
            play_click_sound(game);
            choose_mode(game, CLASSIC_MODE, now);
        }
        else if (key_typed(NUM_3_KEY))
        {
            play_click_sound(game);
            choose_mode(game, RUSH_MODE, now);
        }
        else if (key_typed(NUM_4_KEY))
        {
            play_click_sound(game);
            choose_mode(game, RECIPE_MODE, now);
        }
        return;
    }

    if (game.screen == PLAYING_SCREEN && (key_typed(P_KEY) || key_typed(ESCAPE_KEY)))
    {
        play_click_sound(game);
        toggle_pause(game, now);
        return;
    }

    if (game.paused)
    {
        if (key_typed(R_KEY))
            show_mode_menu(game);
        return;
    }

```

```
    if (key_typed(SPACE_KEY))
    {
        if (game.screen == STORY_SCREEN)
            advance_story_with_voice(game, now);
        else
            advance_screen(game, now);
    }

    // R returns to the four modes (never back to the story).
    if (key_typed(R_KEY) && game.screen != STORY_SCREEN)
    {
        show_mode_menu(game);
    }
}

// ========= File: src/game_view.h ==========
// src/game_view.h
#ifndef GAME_VIEW_H
#define GAME_VIEW_H

#include "splashkit.h"
#include "game_state.h"

void draw_game(const GameState &game, unsigned int now);
void load_game_art();

#endif

// ========= File: src/game_view.cpp ==========
// src/game_view.cpp
#include "splashkit.h"
#include "game_view.h"
#include "sound_manager.h"
#include "score_manager.h"
#include "game_logic.h"
#include "game_layout.h"
#include "utilities.h"

// Keep the colour creation in one place so the rest of the file stays readable.
static color rgb(int r, int g, int b)
{
    return rgb_color(r, g, b);
}

static const string UI_FONT_NAME = "sweetstack_ui";
static const int UI_FONT_SIZE = 14;
static bool ui_font_loaded = false;

static void draw_game_text(const string &text, const color &colour, double x, double y)
{
    if (ui_font_loaded)
        draw_text(text, colour, UI_FONT_NAME, UI_FONT_SIZE, x, y);
    else
        draw_text(text, colour, x, y);
}

```

```
static int centered_ui_text_x(const string &text, int box_x, int box_width)
{
    if (ui_font_loaded)
        return box_x + (box_width - text_width(text, UI_FONT_NAME, UI_FONT_SIZE)) / 2;
    return centered_text_x(text, box_x, box_width);
}

// Return a readable name for each food chain.
static string item_name(ItemCategory category)
{
    if (category == DOUGH)
    {
        return "Dough";
    }
    else if (category == TART)
    {
        return "Tart";
    }
    else if (category == CAKE)
    {
        return "Cake";
    }

    return "food";
}

// Load one picture and tell the player in the terminal if the file was not found.
static void load_picture(const string &name, const string &path)
{
    load_bitmap(name, path);

    if (!has_bitmap(name))
    {
        write_line("Could not load picture: " + path);
        write_line("Run the game from inside the SweetStack folder.");
    }
}

// Load the background, food pictures, shop owner, and customer portraits.
void load_game_art()
{
    load_font(UI_FONT_NAME, "Resources/fonts/Coiny-Regular.ttf");
    ui_font_loaded = has_font(UI_FONT_NAME);
    if (!ui_font_loaded)
        write_line("Could not load font: Resources/fonts/Coiny-Regular.ttf");

    load_picture("bakery_background", "Resources/images/background.png");
    load_picture("shop_owner", "Resources/images/shop_owner_story.png");
    for (int row = 0; row < 3; row++)
    {
        for (int column = 0; column < 4; column++)
        {
            string name = "food_" + number_text(row) + "_" + number_text(column);
            load_picture(name, "Resources/images/food/" + name + ".png");
        }
    }
```

```
1608
1609        // generator_0 = dough mixer, generator_1 = tart oven, generator_2 = cake mixer.
1610        for (int i = 0; i < 3; i++)
1611        {
1612            string name = "generator_" + number_text(i);
1613            load_picture(name, "Resources/images/food/" + name + ".png");
1614        }
1615
1616        // The customer portraits used on the play screen.
1617        for (int i = 1; i <= 5; i++)
1618        {
1619            string file = "customer_0" + number_text(i);
1620            load_picture("customer_" + number_text(i),
1621                "Resources/images/customers/" + file + ".png");
1622        }
1623        load_picture("customer_story_1", "Resources/images/customers/customer_01_story.png");
1624    }
1625
1626    // Convert the current mode into a label for the screen.
1627    static string mode_name(GameMode mode)
1628    {
1629        if (mode == BEGINNER_MODE)
1630        {
1631            return "BEGINNER";
1632        }
1633        else if (mode == CLASSIC_MODE)
1634        {
1635            return "CLASSIC BAKERY";
1636        }
1637        else if (mode == RUSH_MODE)
1638        {
1639            return "RUSH HOUR";
1640        }
1641
1642        return "RECIPE CHALLENGE";
1643    }
1644
1645    // Draw a food item from the atlas.
1646    // Each row represents a food category and each column represents its tier.
1647    static void draw_item(ItemCategory category, int tier, int x, int y)
1648    {
1649        if (category == NO_ITEM)
1650        {
1651            return;
1652        }
1653
1654        int atlasRow = 0;
1655        if (category == DOUGH)
1656        {
1657            atlasRow = 0;
1658        }
1659        else if (category == TART)
1660        {
1661            atlasRow = 1;
1662        }
1663        else if (category == CAKE)
```

```
    {
        atlasRow = 2;
    }

    // Higher tiers reuse the final picture.
    int atlasColumn = tier;
    if (atlasColumn > 3)
    {
        atlasColumn = 3;
    }

    if (atlasColumn < 0)
    {
        atlasColumn = 0;
    }

    string name = "food_" + number_text(atlasRow) + "_" + number_text(atlasColumn);
    draw_bitmap(name, x + FOOD_MARGIN, y + FOOD_MARGIN);
}

// Draw a customer or the shop owner at a chosen position.
static void draw_character(const string &name, int x, int y)
{
    if (has_bitmap(name))
    {
        draw_bitmap(name, x, y);
    }
    else
    {
        // The picture is missing: draw a plain marker so the problem is easy to see.
        fill_circle(rgb(211, 93, 127), x + 69, y + 63, 40);
        draw_game_text("?", rgb(255, 255, 255), x + 65, y + 55);
    }
}

// Draw the title bar used by each screen.
static void draw_header(const string &subtitle)
{
    fill_rectangle(rgb(255, 232, 221), 0, 0, WINDOW_WIDTH, 112);
    draw_game_text("SWEET STACK", rgb(121, 69, 84), 54, 22);
    draw_game_text(subtitle, rgb(146, 109, 116), 56, 62);
}

// Draw one mode choice on the mode-selection screen.
static void mode_card(int x, int y, const char *title, const char *lineA, const char *lineB, color
accent)
{
    fill_rectangle(rgb(255, 255, 255), x, y, MODE_CARD_WIDTH, MODE_CARD_HEIGHT);
    draw_rectangle(accent, x, y, MODE_CARD_WIDTH, MODE_CARD_HEIGHT);
    draw_game_text(title, accent, x + 20, y + 16);
    draw_game_text(lineA, rgb(91, 78, 82), x + 20, y + 55);
    draw_game_text(lineB, rgb(126, 116, 116), x + 20, y + 86);
}

// Draw all four game-mode choices.
// Draw one ON/OFF button for the settings panel.
```

```
static void draw_toggle(int y, bool isOn)
{
    color colour = rgb(176, 166, 166);
    string label = "OFF";

    if (isOn)
    {
        colour = rgb(105, 157, 126);
        label = "ON";
    }

    fill_rectangle(colour, SETTINGS_TOGGLE_X, y, SETTINGS_TOGGLE_WIDTH, SETTINGS_TOGGLE_HEIGHT);
    draw_game_text(label, rgb(255, 255, 255),
        centered_ui_text_x(label, SETTINGS_TOGGLE_X, SETTINGS_TOGGLE_WIDTH), y + 12);
}

// Draw the settings panel on top of the mode-selection screen.
static void draw_settings_panel(const GameState &game)
{
    int labelX = SETTINGS_BOX_X + 30;

    // A darker copy offset behind the panel gives it a shadow.
    fill_rectangle(rgb(121, 69, 84), SETTINGS_BOX_X + 6, SETTINGS_BOX_Y + 6,
        SETTINGS_BOX_WIDTH, SETTINGS_BOX_HEIGHT);
    fill_rectangle(rgb(255, 255, 255), SETTINGS_BOX_X, SETTINGS_BOX_Y,
        SETTINGS_BOX_WIDTH, SETTINGS_BOX_HEIGHT);
    draw_rectangle(rgb(121, 69, 84), SETTINGS_BOX_X, SETTINGS_BOX_Y,
        SETTINGS_BOX_WIDTH, SETTINGS_BOX_HEIGHT);

    draw_game_text("SETTINGS", rgb(121, 69, 84),
        centered_ui_text_x("SETTINGS", SETTINGS_BOX_X, SETTINGS_BOX_WIDTH), SETTINGS_BOX_Y + 20);

    draw_game_text("Background music", rgb(91, 78, 82), labelX, SETTINGS_MUSIC_Y + 12);
    draw_toggle(SETTINGS_MUSIC_Y, game.musicOn);

    draw_game_text("Sound effects", rgb(91, 78, 82), labelX, SETTINGS_EFFECTS_Y + 12);
    draw_toggle(SETTINGS_EFFECTS_Y, game.soundEffectsOn);

    draw_game_text("Changes stay until you change them.", rgb(126, 116, 116),
        centered_ui_text_x("Changes stay until you change them.", SETTINGS_BOX_X, SETTINGS_BOX_WIDTH),
        SETTINGS_BOX_Y + 200);

    fill_rectangle(rgb(121, 69, 84), SETTINGS_CLOSE_X, SETTINGS_CLOSE_Y,
        SETTINGS_CLOSE_WIDTH, SETTINGS_CLOSE_HEIGHT);
    draw_game_text("CLOSE", rgb(255, 255, 255),
        centered_ui_text_x("CLOSE", SETTINGS_CLOSE_X, SETTINGS_CLOSE_WIDTH), SETTINGS_CLOSE_Y + 14);
}

static void draw_menu(const GameState &game)
{
    draw_header("Choose how you want to bake");

    fill_rectangle(rgb(121, 69, 84), SETTINGS_BUTTON_X, SETTINGS_BUTTON_Y,
        SETTINGS_BUTTON_WIDTH, SETTINGS_BUTTON_HEIGHT);
    draw_game_text("SETTINGS", rgb(255, 255, 255),
```

```
        centered_ui_text_x("SETTINGS", SETTINGS_BUTTON_X, SETTINGS_BUTTON_WIDTH), SETTINGS_BUTTON_Y +
10);

    mode_card(MODE_LEFT_X, MODE_TOP_Y, "1  BEGINNER",
        "Learn as you play", "One-item orders; no timer", rgb(105, 157, 126));

    mode_card(MODE_RIGHT_X, MODE_TOP_Y, "2  CLASSIC BAKERY",
        "Five random customers", "Orders ask for 1 or 2 items", rgb(116, 139, 184));

    mode_card(MODE_LEFT_X, MODE_BOTTOM_Y, "3  RUSH HOUR",
        "30s per order", "Random; 10s per heart fades", rgb(207, 127, 99));

    mode_card(MODE_RIGHT_X, MODE_BOTTOM_Y, "4  RECIPE CHALLENGE",
        "Limited moves for each order", "Random two-food recipes", rgb(165, 117, 179));

    fill_rectangle(rgb(121, 69, 84), 350, 535, 260, 56);
    draw_game_text("CLICK A MODE (OR PRESS 1-4)", rgb(255, 255, 255), 372, 554);

    if (game.settingsOpen)
        draw_settings_panel(game);
}

// Draw the story scene with the shop owner and the customer.
static void draw_story(const GameState &game)
{
    draw_header("A little story before baking");

    if (game.storyLine == 1)
    {
        draw_character("customer_story_1", 52, 212);
    }
    else
    {
        draw_character("shop_owner", 52, 212);
    }

    // Name plate under the speaker, so the player knows who is talking.
    string speaker = "MIA - SHOP OWNER";
    if (game.storyLine == 1)
        speaker = "FESTIVAL GUEST";

    draw_game_text(speaker, rgb(121, 69, 84),
        centered_ui_text_x(speaker, STORY_NAME_X, STORY_NAME_WIDTH), STORY_NAME_Y + 5);

    fill_rectangle(rgb(255, 255, 255), 295, 232, 570, 184);
    draw_rectangle(rgb(231, 207, 207), 295, 232, 570, 184);
    draw_game_text("Mia's Moonlight Bakery", rgb(121, 69, 84), 325, 253);

    if (game.storyLine == 0)
    {
        draw_game_text("Mia: Hi, I'm Mia, the baker who runs this shop.", rgb(91, 78, 82), 325, 289);
        draw_game_text("The Moonlight Festival is tonight!", rgb(91, 78, 82), 325, 325);
        draw_game_text("Our bakery has just reopened.", rgb(91, 78, 82), 325, 358);
    }
    else if (game.storyLine == 1)
    {
```

```
        draw_game_text("Customer: We need treats for the festival.", rgb(91, 78, 82), 325, 299);
        draw_game_text("Can you help us prepare the orders?", rgb(91, 78, 82), 325, 330);
    }
    else
    {
        draw_game_text("Mia: Choose your challenge and let's bake!", rgb(91, 78, 82), 325, 299);
        draw_game_text("Every successful recipe helps the festival.", rgb(91, 78, 82), 325, 330);
    }

    fill_rectangle(rgb(121, 69, 84), 390, 482, 180, 54);
    draw_game_text("CONTINUE", rgb(255, 255, 255), 435, 500);
    draw_game_text("Click or press SPACE", rgb(255, 255, 255), 395, 555);
}

// Draw one generator button and a small preview of the food it creates.
static void draw_generator_button(const char *label, ItemCategory category, int x)
{
    fill_rectangle(rgb(255, 242, 235), x, GENERATOR_Y, GENERATOR_WIDTH, GENERATOR_HEIGHT);
    draw_rectangle(rgb(201, 157, 145), x, GENERATOR_Y, GENERATOR_WIDTH, GENERATOR_HEIGHT);

    // The generator atlas has three square sprites in one horizontal row.
    int spriteColumn = 0;
    if (category == TART)
    {
        spriteColumn = 1;
    }
    else if (category == CAKE)
    {
        spriteColumn = 2;
    }

    draw_bitmap("generator_" + number_text(spriteColumn), x + 4, GENERATOR_Y + 4);
    draw_game_text(label, rgb(121, 69, 84), x + 58, GENERATOR_Y + 19);
}

// Draw the three generators below the 4 by 4 board.
static void draw_generators()
{
    draw_generator_button("DOUGH", DOUGH, DOUGH_BUTTON_X);
    draw_generator_button("TART", TART, TART_BUTTON_X);
    draw_generator_button("CAKE", CAKE, CAKE_BUTTON_X);
}

// Beginner mode arrows point to a useful matching pair, or the needed generator.
static int draw_beginner_guidance(const GameState &game)
{
    if (game.mode != BEGINNER_MODE || game.orderJustCompleted || game.orderIndex >=
order_count_for_mode(game.mode))
        return 3;

    ItemCategory needed = current_order_category(game);
    int requiredTier = current_order_tier(game);
    int firstRow = -1, firstCol = -1, secondRow = -1, secondCol = -1;

    if (game.selectedRow >= 0 && game.selectedColumn >= 0)
    {
```

```
        BoardItem selected = game.board[game.selectedRow][game.selectedColumn];
        if (selected.category == needed && selected.tier < requiredTier)
        {
            for (int row = 0; row < GRID_SIZE; row++)
            {
                for (int col = 0; col < GRID_SIZE; col++)
                {
                    BoardItem item = game.board[row][col];
                    if ((row != game.selectedRow || col != game.selectedColumn) &&
                        item.category == selected.category && item.tier == selected.tier)
                    {
                        secondRow = row;
                        secondCol = col;
                        break;
                    }
                }
                if (secondRow >= 0) break;
            }
            if (secondRow >= 0)
            {
                int x = BOARD_X + secondCol * (CELL_SIZE + CELL_GAP) + CELL_SIZE / 2;
                int y = BOARD_Y + secondRow * (CELL_SIZE + CELL_GAP);
                draw_line(rgb(221, 142, 66), x, y - 28, x, y - 13);
                fill_triangle(rgb(221, 142, 66), x - 8, y - 17, x + 8, y - 17, x, y - 4);
                return 2;
            }
        }
    }

    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int col = 0; col < GRID_SIZE; col++)
        {
            BoardItem first = game.board[row][col];
            if (first.category != needed || first.tier >= requiredTier) continue;
            for (int otherRow = row; otherRow < GRID_SIZE; otherRow++)
            {
                int startCol = (otherRow == row) ? col + 1 : 0;
                for (int otherCol = startCol; otherCol < GRID_SIZE; otherCol++)
                {
                    BoardItem second = game.board[otherRow][otherCol];
                    if (second.category == first.category && second.tier == first.tier)
                    {
                        firstRow = row; firstCol = col;
                        secondRow = otherRow; secondCol = otherCol;
                        break;
                    }
                }
                if (secondRow >= 0) break;
            }
            if (secondRow >= 0) break;
        }
        if (secondRow >= 0) break;
    }

    if (secondRow >= 0)
```

```
    {
        int firstX = BOARD_X + firstCol * (CELL_SIZE + CELL_GAP) + CELL_SIZE / 2;
        int firstY = BOARD_Y + firstRow * (CELL_SIZE + CELL_GAP);
        int secondX = BOARD_X + secondCol * (CELL_SIZE + CELL_GAP) + CELL_SIZE / 2;
        int secondY = BOARD_Y + secondRow * (CELL_SIZE + CELL_GAP);
        draw_line(rgb(221, 142, 66), firstX, firstY - 28, firstX, firstY - 13);
        fill_triangle(rgb(221, 142, 66), firstX - 8, firstY - 17, firstX + 8, firstY - 17, firstX, firstY - 4);
        draw_line(rgb(221, 142, 66), secondX, secondY - 28, secondX, secondY - 13);
        fill_triangle(rgb(221, 142, 66), secondX - 8, secondY - 17, secondX + 8, secondY - 17, secondX, secondY - 4);
        return 1;
    }

    bool hasEmpty = false;
    for (int row = 0; row < GRID_SIZE; row++)
        for (int col = 0; col < GRID_SIZE; col++)
            if (game.board[row][col].category == NO_ITEM) hasEmpty = true;
    if (!hasEmpty) return 3;

    int buttonX = (needed == DOUGH) ? DOUGH_BUTTON_X : (needed == TART) ? TART_BUTTON_X : CAKE_BUTTON_X;
    int x = buttonX + GENERATOR_WIDTH / 2;
    int y = GENERATOR_Y;
    draw_line(rgb(221, 142, 66), x, y - 28, x, y - 13);
    fill_triangle(rgb(221, 142, 66), x - 8, y - 17, x + 8, y - 17, x, y - 4);
    return 0;
}

// Draw the customer on the left and their order on the right.
static void draw_customer_and_order(const GameState &game, unsigned int now)
{
    bool leaving = customer_has_left(game, now);
    int textX = ORDER_BOX_X + 16;

    // CUSTOMER box and portrait (the portrait is missing once the customer has left).
    fill_rectangle(rgb(255, 255, 255), RIGHT_X, CUSTOMER_BOX_Y, CUSTOMER_BOX_WIDTH, CUSTOMER_BOX_HEIGHT);
    draw_rectangle(rgb(231, 207, 207), RIGHT_X, CUSTOMER_BOX_Y, CUSTOMER_BOX_WIDTH, CUSTOMER_BOX_HEIGHT);

    if (!leaving)
    {
        int customerNumber = (game.orderIndex % 5) + 1;
        draw_character("customer_" + number_text(customerNumber), RIGHT_X + 4, CUSTOMER_BOX_Y + 9);
    }

    // ORDER box, right next to the customer.
    fill_rectangle(rgb(255, 255, 255), ORDER_BOX_X, CUSTOMER_BOX_Y,
        ORDER_BOX_WIDTH, CUSTOMER_BOX_HEIGHT);
    draw_rectangle(rgb(231, 207, 207), ORDER_BOX_X, CUSTOMER_BOX_Y,
        ORDER_BOX_WIDTH, CUSTOMER_BOX_HEIGHT);

    if (leaving)
    {
        draw_game_text("Next customer...", rgb(126, 116, 116), textX, CUSTOMER_BOX_Y + 60);
```

```
        return;
    }

    if (game.orderJustCompleted)
    {
        draw_game_text("ORDER COMPLETE!", rgb(105, 157, 126), textX, CUSTOMER_BOX_Y + 42);
        draw_game_text("Thank you!", rgb(91, 78, 82), textX, CUSTOMER_BOX_Y + 72);
        return;
    }

    draw_game_text("CUSTOMER ORDER", rgb(121, 69, 84), textX, CUSTOMER_BOX_Y + 12);

    string orderText = number_text(current_order_quantity(game));
    orderText += " x level ";
    orderText += number_text(current_order_tier(game) + 1);
    orderText += " ";
    orderText += item_name(current_order_category(game));

    draw_game_text(orderText, rgb(91, 78, 82), textX, CUSTOMER_BOX_Y + 42);

    int firstReady = game.deliveredCount;
    if (game.mode == RECIPE_MODE)
    {
        firstReady = game.recipeFirstDelivered;
    }

    string readyText = "Ready: ";
    readyText += number_text(firstReady);
    readyText += " / ";
    readyText += number_text(current_order_quantity(game));

    draw_game_text(readyText, rgb(126, 116, 116), textX, CUSTOMER_BOX_Y + 68);

    if (game.mode == RECIPE_MODE)
    {
        string secondOrderText = "+ ";
        secondOrderText += number_text(current_order_secondary_quantity(game));
        secondOrderText += " x level ";
        secondOrderText += number_text(current_order_secondary_tier(game) + 1);
        secondOrderText += " ";
        secondOrderText += item_name(current_order_secondary_category(game));

        draw_game_text(secondOrderText, rgb(91, 78, 82), textX, CUSTOMER_BOX_Y + 96);

        string secondReadyText = "Ready: ";
        secondReadyText += number_text(game.recipeSecondDelivered);
        secondReadyText += " / ";
        secondReadyText += number_text(current_order_secondary_quantity(game));

        draw_game_text(secondReadyText, rgb(126, 116, 116), textX, CUSTOMER_BOX_Y + 120);
    }
}

// Draw the progress bar used for the Rush Hour timer and the Recipe moves.
static void draw_bar(color barColour, int x, int y, double fraction)
{
```

```
    if (fraction < 0) fraction = 0;
    if (fraction > 1) fraction = 1;

    fill_rectangle(rgb(245, 229, 225), x, y, 260, 19);
    fill_rectangle(barColour, x, y, (int)(260.0 * fraction), 19);
}

// The panel under the order. It shows what matters in the chosen mode:
//   Rush Hour - the customer's hearts and the 60 second timer
//   Recipe - how many moves are left
//   Beginner/Classic - customer number and no timer.
static void draw_status_panel(const GameState &game, unsigned int now)
{
    int textX = RIGHT_X + 20;
    fill_rectangle(rgb(255, 255, 255), RIGHT_X, STATUS_BOX_Y, RIGHT_WIDTH, STATUS_BOX_HEIGHT);
    draw_rectangle(rgb(236, 218, 218), RIGHT_X, STATUS_BOX_Y, RIGHT_WIDTH, STATUS_BOX_HEIGHT);

    if (game.mode == RUSH_MODE)
    {
        draw_game_text("CUSTOMER SATISFACTION", rgb(121, 69, 84), textX, STATUS_BOX_Y + 15);

        for (int heart = 0; heart < MAX_HEARTS; heart++)
        {
            color heartColour = rgb(221, 211, 211);
            if (heart < game.satisfaction)
                heartColour = rgb(229, 87, 116);
            fill_circle(heartColour, RIGHT_X + 39 + heart * 36, STATUS_BOX_Y + 59, 12);
        }

        unsigned int left = current_order_time_left(game, now);
        color timerColour = rgb(228, 167, 101);
        if (left < 15000)
            timerColour = rgb(221, 94, 103);

        draw_bar(timerColour, textX, STATUS_BOX_Y + 86, (double)left / ORDER_TIME_LIMIT);
        draw_game_text(number_text(left / 1000) + " seconds left", rgb(91, 78, 82), textX, STATUS_BOX_Y
+ 112);
    }
    else if (game.mode == RECIPE_MODE)
    {
        draw_game_text("MOVES LEFT", rgb(121, 69, 84), textX, STATUS_BOX_Y + 15);
        draw_game_text(number_text(game.movesLeft) + " of " + number_text(current_move_limit(game)) + "
moves", rgb(91, 78, 82), textX, STATUS_BOX_Y + 48);

        color movesColour = rgb(165, 117, 179);
        if (game.movesLeft <= 5)
            movesColour = rgb(221, 94, 103);

        draw_bar(movesColour, textX, STATUS_BOX_Y + 78, (double)game.movesLeft /
current_move_limit(game));
        draw_game_text("Out of moves ends the game.", rgb(126, 116, 116), textX, STATUS_BOX_Y + 106);
    }
    else
    {
        draw_game_text("CUSTOMER " + number_text(game.orderIndex + 1) + " OF " +
number_text(order_count_for_mode(game.mode)), rgb(121, 69, 84), textX, STATUS_BOX_Y + 15);
```

```
        draw_game_text("No timer in this mode.", rgb(91, 78, 82), textX, STATUS_BOX_Y + 55);
        draw_game_text("Take your time!", rgb(126, 116, 116), textX, STATUS_BOX_Y + 85);
    }
}

// Show the selected food and its requested level in every game mode.
static void draw_selection_panel(const GameState &game)
{
    fill_rectangle(rgb(255, 255, 255), RIGHT_X, 470, RIGHT_WIDTH, 130);
    draw_rectangle(rgb(236, 218, 218), RIGHT_X, 470, RIGHT_WIDTH, 130);
    draw_game_text("SELECTED FOOD", rgb(121, 69, 84), RIGHT_X + 20, 482);

    if (game.selectedRow >= 0 && game.selectedColumn >= 0)
    {
        BoardItem selected = game.board[game.selectedRow][game.selectedColumn];
        draw_item(selected.category, selected.tier, RIGHT_X + 8, 488);
        draw_game_text(item_name(selected.category), rgb(91, 78, 82), RIGHT_X + 126, 502);
        draw_game_text("Level " + number_text(selected.tier + 1), rgb(91, 78, 82), RIGHT_X + 126, 528);
        int neededTier = -1;
        if (selected.category == current_order_category(game))
            neededTier = current_order_tier(game);
        else if (game.mode == RECIPE_MODE && selected.category == current_order_second-
ary_category(game))
            neededTier = current_order_secondary_tier(game);

        if (neededTier >= 0)
            draw_game_text("Order needs level " + number_text(neededTier + 1) + ".", rgb(126, 116,
116), RIGHT_X + 126, 554);
        else
            draw_game_text("Not part of this order.", rgb(126, 116, 116), RIGHT_X + 126, 554);
    }
    else if (game.lastAction != "")
    {
        draw_game_text(game.lastAction, rgb(91, 78, 82), RIGHT_X + 20, 528);
    }
    else
    {
        draw_game_text("Nothing selected.", rgb(91, 78, 82), RIGHT_X + 20, 522);
        draw_game_text("Click a food tile on the tray to inspect it.", rgb(126, 116, 116), RIGHT_X +
20, 550);
    }
}

// Draw the game board, customer order, and status information.
static void draw_play(const GameState &game, unsigned int now)
{
    draw_header(mode_name(game.mode));

    // Keep a visible way back to the mode menu during play.
    fill_rectangle(rgb(121, 69, 84), BACK_X, BACK_Y, BACK_WIDTH, BACK_HEIGHT);
    draw_game_text("BACK", rgb(255, 255, 255), BACK_X + 33, BACK_Y + 10);

    draw_game_text("SCORE  " + number_text(game.score), rgb(121, 69, 84), 690, 25);
    draw_game_text("MOVES  " + number_text(game.moves), rgb(121, 69, 84), 690, 66);

```

```
    fill_rectangle(rgb(121, 69, 84), PAUSE_BUTTON_X, PAUSE_BUTTON_Y, PAUSE_BUTTON_WIDTH, PAUSE_BUTTON_HEIGHT);
    string pauseLabel = game.paused ? "RESUME" : "PAUSE";
    draw_game_text(pauseLabel, rgb(255, 255, 255), centered_ui_text_x(pauseLabel, PAUSE_BUTTON_X, PAUSE_BUTTON_WIDTH), PAUSE_BUTTON_Y + 10);

    draw_game_text("BAKERY TRAY", rgb(121, 69, 84), BOARD_X, 130);

    for (int row = 0; row < GRID_SIZE; row++)
    {
        for (int col = 0; col < GRID_SIZE; col++)
        {
            int x = BOARD_X + col * (CELL_SIZE + CELL_GAP);
            int y = BOARD_Y + row * (CELL_SIZE + CELL_GAP);
            fill_rectangle(rgb(255, 255, 255), x, y, CELL_SIZE, CELL_SIZE);
            draw_rectangle(rgb(239, 224, 220), x, y, CELL_SIZE, CELL_SIZE);

            BoardItem item = game.board[row][col];
            draw_item(item.category, item.tier, x, y);
            if (row == game.selectedRow && col == game.selectedColumn)
            {
                fill_rectangle(rgba_color(211, 93, 127, 45), x, y, CELL_SIZE, CELL_SIZE);
                draw_rectangle(rgb(211, 93, 127), x + 2, y + 2, CELL_SIZE - 4, CELL_SIZE - 4);
                draw_rectangle(rgb(211, 93, 127), x + 4, y + 4, CELL_SIZE - 8, CELL_SIZE - 8);
            }
        }
    }

    draw_generators();
    draw_beginner_guidance(game);
    draw_customer_and_order(game, now);
    draw_status_panel(game, now);

    draw_selection_panel(game);

    if (game.paused)
    {
        fill_rectangle(rgba_color(54, 42, 47, 155), 0, 112, WINDOW_WIDTH, WINDOW_HEIGHT - 112);
        fill_rectangle(rgb(255, 247, 241), PAUSE_PANEL_X, PAUSE_PANEL_Y, PAUSE_PANEL_WIDTH, PAUSE_PANEL_HEIGHT);
        draw_rectangle(rgb(211, 93, 127), PAUSE_PANEL_X, PAUSE_PANEL_Y, PAUSE_PANEL_WIDTH, PAUSE_PANEL_HEIGHT);
        draw_game_text("PAUSED", rgb(121, 69, 84), centered_ui_text_x("PAUSED", PAUSE_PANEL_X, PAUSE_PANEL_WIDTH), PAUSE_PANEL_Y + 32);
        draw_game_text("Click RESUME or press P to continue.", rgb(91, 78, 82), centered_ui_text_x("Click RESUME or press P to continue.", PAUSE_PANEL_X, PAUSE_PANEL_WIDTH), PAUSE_PANEL_Y + 78);
    }
}

// Draw one centred line of text inside the end-screen panel.
static void draw_end_line(const string &text, color colour, int y)
{
    draw_game_text(text, colour, centered_ui_text_x(text, END_BOX_X, END_BOX_WIDTH), y);
}

// Draw the win or game-over screen on a white panel so the text is easy to read.
```

```
2201 static void draw_end(const GameState &game)
2202 {
2203     draw_header(mode_name(game.mode) + " RESULTS");
2204     fill_rectangle(rgb(255, 255, 255), END_BOX_X, END_BOX_Y, END_BOX_WIDTH, END_BOX_HEIGHT);
2205     draw_rectangle(rgb(231, 207, 207), END_BOX_X, END_BOX_Y, END_BOX_WIDTH, END_BOX_HEIGHT);
2206 
2207     if (game.screen == WON_SCREEN)
2208     {
2209         draw_end_line("FESTIVAL SAVED!", rgb(105, 157, 126), END_BOX_Y + 36);
2210         draw_end_line("You served every order at Mia's bakery.", rgb(91, 78, 82), END_BOX_Y + 86);
2211     }
2212     else if (game.lostByTimer)
2213     {
2214         draw_end_line("THE CUSTOMER LEFT", rgb(164, 78, 89), END_BOX_Y + 36);
2215         draw_end_line("They waited too long for their order.", rgb(91, 78, 82), END_BOX_Y + 86);
2216     }
2217     else if (game.lostByMoveLimit)
2218     {
2219         draw_end_line("OUT OF MOVES", rgb(164, 78, 89), END_BOX_Y + 36);
2220         draw_end_line("You ran out of moves for this recipe.", rgb(91, 78, 82), END_BOX_Y + 86);
2221     }
2222     else
2223     {
2224         draw_end_line("NO MOVES LEFT", rgb(164, 78, 89), END_BOX_Y + 36);
2225         draw_end_line("The board is full and nothing can merge.", rgb(91, 78, 82), END_BOX_Y + 86);
2226     }
2227 
2228 
2229     draw_end_line("Final score: " + number_text(game.score), rgb(91, 78, 82), END_BOX_Y + 126);
2230     if (game.mode != BEGINNER_MODE)
2231     {
2232         if (game.newHighScore)
2233             draw_end_line("NEW PERSONAL HIGHEST MARK!", rgb(105, 157, 126), END_BOX_Y + 157);
2234         else
2235             draw_end_line("Personal highest mark: " + number_text(high_score_for_mode(game.mode)),
    rgb(146, 109, 116), END_BOX_Y + 157);
2236     }
2237 
2238     fill_rectangle(rgb(121, 69, 84), END_REPLAY_BUTTON_X, END_BUTTON_Y, END_BUTTON_WIDTH,
    END_BUTTON_HEIGHT);
2239     draw_rectangle(rgb(121, 69, 84), END_REPLAY_BUTTON_X, END_BUTTON_Y, END_BUTTON_WIDTH,
    END_BUTTON_HEIGHT);
2240     draw_game_text("PLAY AGAIN", rgb(255, 255, 255), centered_ui_text_x("PLAY AGAIN",
    END_REPLAY_BUTTON_X, END_BUTTON_WIDTH), END_BUTTON_Y + 13);
2241 
2242     fill_rectangle(rgb(214, 171, 157), END_MENU_BUTTON_X, END_BUTTON_Y, END_BUTTON_WIDTH,
    END_BUTTON_HEIGHT);
2243     draw_rectangle(rgb(214, 171, 157), END_MENU_BUTTON_X, END_BUTTON_Y, END_BUTTON_WIDTH,
    END_BUTTON_HEIGHT);
2244     draw_game_text("BACK TO MENU", rgb(255, 255, 255), centered_ui_text_x("BACK TO MENU",
    END_MENU_BUTTON_X, END_BUTTON_WIDTH), END_BUTTON_Y + 13);
2245 }
2246 
2247 // Draw the background and whichever screen is currently active.
2248 void draw_game(const GameState &game, unsigned int now)
2249 {
2250     clear_screen(rgb(255, 247, 241));
```

```
    // The background picture is already 960 x 700, the same size as the window.
    draw_bitmap("bakery_background", 0, 0);
    if (game.screen == MODE_MENU)
    {
        draw_menu(game);
    }
    else if (game.screen == STORY_SCREEN)
    {
        draw_story(game);
    }
    else if (game.screen == PLAYING_SCREEN)
    {
        draw_play(game, now);
    }
    else
    {
        draw_end(game);
    }
}

// ========= File: src/main.cpp ==========
// src/main.cpp
#include "splashkit.h"
#include "game_state.h"
#include "game_logic.h"
#include "game_view.h"
#include "input.h"
#include "sound_manager.h"
#include "score_manager.h"
#include "game_layout.h"

static void record_round_if_finished(GameState &game)
{
    bool hasEnded = (game.screen == WON_SCREEN || game.screen == GAME_OVER_SCREEN);
    if (hasEnded && !game.resultRecorded)
    {
        game.newHighScore = record_high_score(game.mode, game.score);
        game.resultRecorded = true;
    }
}

int main()
{
    GameState game;
    start_new_game(game);
    open_window("Sweet Stack - Cake Merge", WINDOW_WIDTH, WINDOW_HEIGHT);
    load_game_art();
    load_game_sounds();
    play_story_voice(game, game.storyLine);
    load_background_music();
    play_background_music();

    while (!quit_requested())
    {
        // Handle input, update game rules, then draw the current screen.
        process_events();
```

```
2307
2308        unsigned int now = current_ticks();
2309        update_game(game, now);
2310        record_round_if_finished(game);
2311        handle_key_presses(game, now);
2312        if (mouse_clicked(LEFT_BUTTON))
2313        {
2314            handle_mouse_click(game, mouse_x(), mouse_y(), now);
2315        }
2316        record_round_if_finished(game);
2317        draw_game(game, now);
2318        refresh_screen(60);
2319    }
2320
2321    close_all_windows();
2322    return 0;
2323 }
2324
2325 // ========= File: tests/test_sweet_stack.cpp ==========
2326 // tests/test_sweet_stack.cpp
2327 #include "splashkit.h"
2328 #include "game_state.h"
2329 #include "game_logic.h"
2330 #include "sound_manager.h"
2331 #include "score_manager.h"
2332 #include "game_layout.h"
2333 #include "utilities.h"
2334 #include "input.h"
2335
2336 // Print PASS or FAIL for one check.
2337 void check(bool condition, string test_name)
2338 {
2339     if (condition)
2340     {
2341         write_line("PASS: " + test_name);
2342     }
2343     else
2344     {
2345         write_line("FAIL: " + test_name);
2346     }
2347 }
2348
2349 // Count how many board squares hold food.
2350 int count_items(const GameState &game)
2351 {
2352     int count = 0;
2353     for (int row = 0; row < GRID_SIZE; row++)
2354         for (int col = 0; col < GRID_SIZE; col++)
2355             if (game.board[row][col].category != NO_ITEM)
2356                 count++;
2357     return count;
2358 }
2359
2360 // Put one item on the board by hand.
2361 void place(GameState &game, int row, int col, ItemCategory category, int tier)
2362 {
```

```
2363      game.board[row][col].category = category;
2364      game.board[row][col].tier = tier;
2365  }
2366
2367  void test_screen_flow()
2368  {
2369      GameState game = GameState();   // every value starts at zero
2370      start_new_game(game);
2371      check(game.screen == STORY_SCREEN, "program starts on the story screen");
2372
2373      advance_screen(game, 0);
2374      advance_screen(game, 0);
2375      check(game.screen == STORY_SCREEN, "story is still showing before the last line is finished");
2376      advance_screen(game, 0);
2377      check(game.screen == MODE_MENU, "finishing the story shows the mode menu");
2378
2379      choose_mode(game, RUSH_MODE, 1000);
2380      check(game.screen == PLAYING_SCREEN, "choosing a mode starts gameplay immediately");
2381      check(game.mode == RUSH_MODE, "chosen mode is stored");
2382      check(game.orderStartedAt == 1000, "timer starts when the mode starts");
2383
2384      show_mode_menu(game);
2385      check(game.screen == MODE_MENU, "Back returns to the mode menu (not the story)");
2386
2387      choose_mode(game, RECIPE_MODE, 2000);
2388      check(game.screen == PLAYING_SCREEN && game.movesLeft == 12, "choosing another mode starts it
      immediately with its own settings");
2389  }
2390
2391  void test_generators()
2392  {
2393      GameState game = GameState();   // every value starts at zero
2394      choose_mode(game, BEGINNER_MODE, 0);
2395
2396      check(generate_item(game, DOUGH), "dough generator succeeds");
2397      check(count_items(game) == 1, "dough generator adds exactly one item");
2398
2399      check(generate_item(game, TART), "tart generator succeeds");
2400      check(generate_item(game, CAKE), "cake generator succeeds");
2401      check(count_items(game) == 3, "three generator clicks give three items");
2402
2403      bool allTierZero = true;
2404      for (int row = 0; row < GRID_SIZE; row++)
2405          for (int col = 0; col < GRID_SIZE; col++)
2406              if (game.board[row][col].category != NO_ITEM && game.board[row][col].tier != 0)
2407                  allTierZero = false;
2408      check(allTierZero, "new items start at tier 0");
2409
2410      for (int i = 0; i < 13; i++)
2411          generate_item(game, DOUGH);
2412      check(count_items(game) == GRID_SIZE * GRID_SIZE, "board can be filled completely");
2413      check(!generate_item(game, DOUGH), "generator fails when the board is full");
2414  }
2415
2416  void test_move_and_merge()
2417  {
```

```
    GameState game = GameState();   // every value starts at zero
    choose_mode(game, CLASSIC_MODE, 0);

    place(game, 0, 0, DOUGH, 0);
    select_or_move_cake(game, 0, 0, 0);
    select_or_move_cake(game, 2, 2, 0);
    check(game.board[2][2].category == DOUGH && game.board[0][0].category == NO_ITEM, "item moves into an empty square");
    check(game.moves == 1, "a move is counted");

    place(game, 0, 0, DOUGH, 0);
    select_or_move_cake(game, 0, 0, 0);
    select_or_move_cake(game, 2, 2, 0);
    check(game.board[2][2].tier == 1 && game.board[0][0].category == NO_ITEM, "two matching items merge into a higher tier");
    check(game.score == 10, "merge adds score");

    place(game, 0, 0, TART, 1);
    select_or_move_cake(game, 0, 0, 0);
    select_or_move_cake(game, 2, 2, 0);
    check(game.board[2][2].category == DOUGH && game.board[0][0].category == TART, "different foods do not merge");
}

// Remove every food from the board so a test starts from a known position.
void empty_board(GameState &game)
{
    for (int row = 0; row < 4; row++)
    {
        for (int column = 0; column < 4; column++)
        {
            game.board[row][column].category = NO_ITEM;
            game.board[row][column].tier = 0;
        }
    }
}

// Finish the current Beginner/Classic/Rush order by merging two foods
// that are one tier lower than the tier the customer asked for.
void finish_current_order(GameState &game, unsigned int now)
{
    ItemCategory food = current_order_category(game);
    int tier = current_order_tier(game);

    empty_board(game);
    game.deliveredCount = current_order_quantity(game) - 1;
    place(game, 0, 0, food, tier - 1);
    place(game, 0, 1, food, tier - 1);
    select_or_move_cake(game, 0, 0, now);
    select_or_move_cake(game, 0, 1, now);
}

// Move one food from one square to another empty square.
void move_food(GameState &game, int fromRow, int fromColumn, int toRow, int toColumn)
{
    select_or_move_cake(game, fromRow, fromColumn, 0);
```

```
2471      select_or_move_cake(game, toRow, toColumn, 0);
2472  }
2473
2474  void test_order_complete()
2475  {
2476      GameState game = GameState();   // every value starts at zero
2477      choose_mode(game, BEGINNER_MODE, 0);
2478
2479      // First order needs one tier 2 dough. Merge two tier 1 dough to make it.
2480      place(game, 0, 0, DOUGH, 1);
2481      place(game, 0, 1, DOUGH, 1);
2482      select_or_move_cake(game, 0, 0, 0);
2483      select_or_move_cake(game, 0, 1, 0);
2484      check(game.orderJustCompleted, "making the ordered item completes the order");
2485      check(game.score == 120, "completed order adds the order bonus plus merge score");
2486  }
2487
2488  void test_recipe_rejects_higher_tier()
2489  {
2490      GameState game = GameState();
2491      choose_mode(game, RECIPE_MODE, 0);
2492
2493      game.orders[0].category = DOUGH;
2494      game.orders[0].tier = 1;
2495      game.orders[0].quantity = 1;
2496      game.orders[0].secondaryCategory = CAKE;
2497      game.orders[0].secondaryTier = 1;
2498      game.orders[0].secondaryQuantity = 1;
2499      game.orders[0].moveLimit = 24;
2500      game.movesLeft = 24;
2501
2502      empty_board(game);
2503      place(game, 0, 0, DOUGH, 1);
2504      place(game, 0, 1, DOUGH, 1);
2505      select_or_move_cake(game, 0, 0, 0);
2506      select_or_move_cake(game, 0, 1, 0);
2507      check(game.recipeFirstDelivered == 0 && !game.orderJustCompleted,
2508            "a higher-tier merge does not satisfy a lower-tier recipe item");
2509
2510      empty_board(game);
2511      place(game, 0, 0, DOUGH, 0);
2512      place(game, 0, 1, DOUGH, 0);
2513      select_or_move_cake(game, 0, 0, 0);
2514      select_or_move_cake(game, 0, 1, 0);
2515      check(game.recipeFirstDelivered == 1 && !game.orderJustCompleted,
2516            "an exact-tier recipe merge advances only its matching item");
2517  }
2518
2519  void test_recipe_food_collection_delay()
2520  {
2521      GameState game = GameState();
2522      choose_mode(game, RECIPE_MODE, 0);
2523
2524      game.orders[0].category = DOUGH;
2525      game.orders[0].tier = 1;
2526      game.orders[0].quantity = 1;
```

```
    game.orders[0].secondaryCategory = CAKE;
    game.orders[0].secondaryTier = 1;
    game.orders[0].secondaryQuantity = 1;

    empty_board(game);
    place(game, 0, 0, DOUGH, 0);
    place(game, 0, 1, DOUGH, 0);
    select_or_move_cake(game, 0, 0, 2000);
    select_or_move_cake(game, 0, 1, 2000);

    check(game.recipeFirstDelivered == 1 && !game.orderJustCompleted,
          "Recipe Challenge counts one matching requirement independently");
    check(game.board[0][1].category == DOUGH && game.recipeFoodPending[0][1],
          "delivered recipe food remains visible during collection delay");

    int movesAfterDelivery = game.moves;
    select_or_move_cake(game, 0, 1, 2500);
    check(game.board[0][1].category == DOUGH && game.moves == movesAfterDelivery,
          "food being collected cannot be selected or merged again");

    update_game(game, 2000 + ORDER_COMPLETE_MESSAGE_TIME - 1);
    check(game.board[0][1].category == DOUGH,
          "recipe food remains until the normal one-second collection time");
    update_game(game, 2000 + ORDER_COMPLETE_MESSAGE_TIME);
    check(game.board[0][1].category == NO_ITEM && !game.recipeFoodPending[0][1],
          "recipe food is collected after the normal one-second delay");
}

void test_beginner_has_no_high_mark()
{
    check(high_score_for_mode(BEGINNER_MODE) == 0,
          "Beginner does not display a personal highest mark");
    check(!record_high_score(BEGINNER_MODE, 999999),
          "Beginner scores are not recorded as personal highest marks");
}
void test_customer_leaving()
{
    GameState game = GameState();   // every value starts at zero
    choose_mode(game, BEGINNER_MODE, 0);

    finish_current_order(game, 1000);
    check(game.orderJustCompleted && game.screen == PLAYING_SCREEN, "finished order starts the customer
transition");
    check(game.orderIndex == 0, "order number stays the same until the next customer arrives");
    check(!customer_has_left(game, 1500), "customer is still shown during the Order complete message");
    check(customer_has_left(game, 1000 + ORDER_COMPLETE_MESSAGE_TIME), "customer has left after the
message");

    // The board is locked, so no second order can be completed by accident.
    int movesBefore = game.moves;
    empty_board(game);
    place(game, 2, 0, DOUGH, 0);
    place(game, 2, 1, DOUGH, 0);
    select_or_move_cake(game, 2, 0, 1200);
    select_or_move_cake(game, 2, 1, 1200);
```

```
    check(game.board[2][0].category == DOUGH && game.board[2][1].category == DOUGH,"board is locked while the customer is leaving");
    check(game.moves == movesBefore, "locked clicks do not count as moves");

    update_game(game, 1000 + CUSTOMER_TRANSITION_TIME - 1);
    check(game.orderJustCompleted && game.orderIndex == 0, "next customer does not arrive too early");

    update_game(game, 1000 + CUSTOMER_TRANSITION_TIME);
    check(!game.orderJustCompleted && game.orderIndex == 1, "next customer arrives with the next order");
    check(game.deliveredCount == 0, "new order starts with no progress");

    select_or_move_cake(game, 2, 0, 5000);
    select_or_move_cake(game, 2, 1, 5000);
    check(game.board[2][1].tier == 1, "board works again for the next customer");
}

void test_final_order_wins()
{
    GameState game = GameState();   // every value starts at zero
    choose_mode(game, CLASSIC_MODE, 0);
    unsigned int now = 0;

    for (int i = 0; i < order_count_for_mode(game.mode) - 1; i++)
    {
        finish_current_order(game, now);
        update_game(game, now + CUSTOMER_TRANSITION_TIME);
        now += 10000;
    }
    check(game.orderIndex == order_count_for_mode(game.mode) - 1 && game.screen == PLAYING_SCREEN, "first two orders each bring a new customer");

    finish_current_order(game, now);
    check(game.screen == WON_SCREEN, "final order shows the win screen");
    check(!game.orderJustCompleted, "final order has no customer transition");
}

void test_rush_hour_timer()
{
    check(ORDER_TIME_LIMIT == 30000, "Rush Hour gives each customer 30 seconds");
    check(CUSTOMER_HEART_TIME == 10000, "a heart fades every 10 seconds");
    check(MAX_HEARTS * CUSTOMER_HEART_TIME == ORDER_TIME_LIMIT, "three hearts last exactly one order");

    GameState game = GameState();
    choose_mode(game, RUSH_MODE, 0);
    check(game.satisfaction == 3, "Rush Hour customer starts with three hearts");
    check(current_order_time_left(game, 5000) == 25000, "timer counts down from 30 seconds");

    // One heart is lost every 10 seconds (the last heart goes when time is up).
    update_game(game, 9999);
    check(game.satisfaction == 3, "still three hearts just before 10 seconds");
    update_game(game, 10000);
    check(game.satisfaction == 2, "first heart is lost at 10 seconds");
    update_game(game, 19999);
    check(game.satisfaction == 2, "still two hearts just before 20 seconds");
    update_game(game, 20000);
```

```
2633      check(game.satisfaction == 1, "second heart is lost at 20 seconds");
2634      update_game(game, 29999);
2635      check(game.screen == PLAYING_SCREEN && game.satisfaction == 1, "customer waits for the full 30
          seconds");
2636      update_game(game, ORDER_TIME_LIMIT);
2637      check(game.screen == GAME_OVER_SCREEN && game.lostByTimer, "customer leaves at 30 seconds and the
          game is over");
2638
2639      // Hearts and timer are frozen while the customer is leaving.
2640      choose_mode(game, RUSH_MODE, 0);
2641      finish_current_order(game, 15000);
2642      check(game.satisfaction == 2, "customer keeps the hearts they had when served");
2643      update_game(game, 15500);
2644      check(game.satisfaction == 2, "hearts do not change while the customer is leaving");
2645      check(current_order_time_left(game, 95000) == ORDER_TIME_LIMIT - 15000, "timer is frozen during the
          transition");
2646
2647      // A late order must not be punished by the transition time.
2648      choose_mode(game, RUSH_MODE, 0);
2649      finish_current_order(game, 29000);
2650      update_game(game, 30500);
2651      check(game.screen == PLAYING_SCREEN, "no game over while the customer is leaving");
2652
2653      // The next customer gets full hearts and a full timer.
2654      update_game(game, 29000 + CUSTOMER_TRANSITION_TIME);
2655      check(game.satisfaction == 3, "next customer starts with three hearts");
2656      check(current_order_time_left(game, 29000 + CUSTOMER_TRANSITION_TIME) == ORDER_TIME_LIMIT, "next
          customer gets a full timer");
2657  }
2658
2659  void test_recipe_move_limit()
2660  {
2661      GameState game = GameState();   // every value starts at zero
2662      choose_mode(game, RECIPE_MODE, 0);
2663      int limit = game.movesLeft;
2664      check(limit == 12, "Recipe Challenge starts with 12 moves for the first customer");
2665
2666      empty_board(game);
2667      place(game, 0, 0, DOUGH, 0);
2668      for (int i = 0; i < limit - 1; i++)
2669      {
2670          if (i % 2 == 0)
2671              move_food(game, 0, 0, 0, 1);
2672          else
2673              move_food(game, 0, 1, 0, 0);
2674      }
2675      check(game.movesLeft == 1 && game.screen == PLAYING_SCREEN, "moves are counted down while the game
          continues");
2676
2677      if ((limit - 1) % 2 == 0)
2678          move_food(game, 0, 0, 0, 1);
2679      else
2680          move_food(game, 0, 1, 0, 0);
2681      check(game.screen == GAME_OVER_SCREEN && game.lostByMoveLimit, "using the last move without
          finishing the recipe ends the game");
2682
2683      // Finishing the order on the very last move must not end the game.
```

```
    choose_mode(game, RECIPE_MODE, 0);
    empty_board(game);
    game.movesLeft = 1;
    game.recipeFirstDelivered = current_order_quantity(game) - 1;
    game.recipeSecondDelivered = current_order_secondary_quantity(game);
    place(game, 0, 0, current_order_category(game), current_order_tier(game) - 1);
    place(game, 0, 1, current_order_category(game), current_order_tier(game) - 1);
    select_or_move_cake(game, 0, 0, 0);
    select_or_move_cake(game, 0, 1, 0);
    check(game.orderJustCompleted && game.screen == PLAYING_SCREEN, "finishing an order on the last move is not a loss");
    update_game(game, ORDER_COMPLETE_MESSAGE_TIME);
    check(game.board[0][1].category == NO_ITEM, "recipe food disappears when the customer leaves");
}

void test_served_food_disappears()
{
    GameState game = GameState();   // every value starts at zero
    choose_mode(game, BEGINNER_MODE, 0);
    finish_current_order(game, 1000);
    place(game, 3, 3, TART, 0);   // a food that is not part of the order

    ItemCategory food = current_order_category(game);
    int tier = current_order_tier(game);

    check(game.board[0][1].category == food && game.board[0][1].tier == tier, "ordered food is still on the board while the customer is thanked");

    update_game(game, 1000 + ORDER_COMPLETE_MESSAGE_TIME - 1);
    check(game.board[0][1].category == food, "food stays until the customer leaves");

    update_game(game, 1000 + ORDER_COMPLETE_MESSAGE_TIME);
    check(game.board[0][1].category == NO_ITEM, "ordered food disappears when the customer leaves");
    check(game.board[3][3].category == TART, "other foods stay on the board");
}

// Beginner, Classic and Recipe have no hearts and no timer.
void test_hearts_only_in_rush_hour()
{
    GameState game = GameState();   // every value starts at zero
    choose_mode(game, BEGINNER_MODE, 0);
    update_game(game, 500000);
    check(game.screen == PLAYING_SCREEN, "Beginner has no timer");

    choose_mode(game, CLASSIC_MODE, 0);
    update_game(game, 500000);
    check(game.screen == PLAYING_SCREEN, "Classic Bakery has no timer");

    choose_mode(game, RECIPE_MODE, 0);
    update_game(game, 500000);
    check(game.screen == PLAYING_SCREEN, "Recipe Challenge has no timer");
    check(current_move_limit(game) == game.movesLeft, "Recipe Challenge starts with the full move limit");
}

void test_back_to_menu()
```

```
{
    GameState game = GameState();   // every value starts at zero
    choose_mode(game, RUSH_MODE, 0);
    finish_current_order(game, 1000);

    show_mode_menu(game);
    check(game.screen == MODE_MENU, "Back returns to the mode menu");

    choose_mode(game, BEGINNER_MODE, 2000);
    check(game.screen == PLAYING_SCREEN && !game.orderJustCompleted && game.orderIndex == 0, "choosing a mode after Back starts a fresh game");
}

void test_utilities()
{
    check(number_text(0) == "0", "number_text of 0");
    check(number_text(1250) == "1250", "number_text of 1250");

    check(is_in_range(5, 1, 10), "5 is inside 1 to 10");
    check(is_in_range(1, 1, 10) && is_in_range(10, 1, 10), "range includes both ends");
    check(!is_in_range(0, 1, 10) && !is_in_range(11, 1, 10), "values outside the range are rejected");

    check(estimated_text_width("Hello") == 5 * TEXT_CHAR_WIDTH, "text width is estimated per character");
    check(centered_text_x("abcd", 100, 100) == 100 + (100 - 4 * TEXT_CHAR_WIDTH) / 2, "text is centred in a box");

    check(point_in_rectangle(10, 10, 10, 10, 50, 20), "top-left corner is inside a rectangle");
    check(point_in_rectangle(59, 29, 10, 10, 50, 20), "bottom-right pixel is inside a rectangle");
    check(!point_in_rectangle(60, 20, 10, 10, 50, 20), "just right of a rectangle is outside");
    check(!point_in_rectangle(9, 20, 10, 10, 50, 20), "just left of a rectangle is outside");
}

// The screen layout is shared by drawing and clicking, so check it makes sense.
void test_layout()
{
    check(BOARD_SIZE == 4 * CELL_SIZE + 3 * CELL_GAP, "board size matches cells and gaps");
    check(BOARD_Y + BOARD_SIZE <= GENERATOR_Y, "generator buttons start below the board");
    check(GENERATOR_Y + GENERATOR_HEIGHT <= WINDOW_HEIGHT, "generator buttons fit inside the window");
    check(DOUGH_BUTTON_X + GENERATOR_WIDTH <= TART_BUTTON_X, "dough and tart buttons do not overlap");
    check(TART_BUTTON_X + GENERATOR_WIDTH <= CAKE_BUTTON_X, "tart and cake buttons do not overlap");
    check(CAKE_BUTTON_X + GENERATOR_WIDTH <= BOARD_X + BOARD_SIZE, "buttons are no wider than the board");
    check(FOOD_IMAGE_SIZE + 2 * FOOD_MARGIN == CELL_SIZE, "food picture is centred in its square");
    check(GENERATOR_ICON_SIZE + 8 <= GENERATOR_HEIGHT, "generator icon fits inside its button");
    check(BOARD_X + BOARD_SIZE < RIGHT_X, "board does not overlap the right-hand panels");
    check(RIGHT_X + CUSTOMER_BOX_WIDTH <= ORDER_BOX_X, "customer box does not overlap the order box");
    check(ORDER_BOX_X + ORDER_BOX_WIDTH <= WINDOW_WIDTH, "order box fits inside the window");
    check(BACK_X + BACK_WIDTH <= WINDOW_WIDTH, "Back button fits inside the window");

    // Text must keep a margin of at least 16 pixels inside its box.
    check(estimated_text_width("Orders ask for 1 or 2 items") + 2 * 20 <= MODE_CARD_WIDTH, "longest mode-card line fits its card");
    check(estimated_text_width("+ 1 x level 4 tart") + 2 * 16 <= ORDER_BOX_WIDTH, "recipe order line fits the order box");
    check(estimated_text_width("ORDER COMPLETE!") + 2 * 16 <= ORDER_BOX_WIDTH, "order-complete message fits the order box");
```

```
    check(estimated_text_width("Click or press SPACE to choose a mode again.") + 2 * 16 <= END_BOX_WIDTH, "end-screen hint fits its panel");
    check(MODE_RIGHT_X + MODE_CARD_WIDTH <= WINDOW_WIDTH, "mode cards fit inside the window");

    // Settings button and panel.
    check(SETTINGS_BUTTON_X + SETTINGS_BUTTON_WIDTH <= WINDOW_WIDTH, "Settings button fits inside the window");
    check(SETTINGS_BOX_X + SETTINGS_BOX_WIDTH <= WINDOW_WIDTH && SETTINGS_BOX_Y + SETTINGS_BOX_HEIGHT <= WINDOW_HEIGHT,
          "settings panel fits inside the window");
    check(SETTINGS_TOGGLE_X + SETTINGS_TOGGLE_WIDTH + 16 <= SETTINGS_BOX_X + SETTINGS_BOX_WIDTH, "ON/OFF buttons fit inside the panel");
    check(SETTINGS_MUSIC_Y + SETTINGS_TOGGLE_HEIGHT <= SETTINGS_EFFECTS_Y, "the two switches do not overlap");
    check(SETTINGS_EFFECTS_Y + SETTINGS_TOGGLE_HEIGHT < SETTINGS_CLOSE_Y, "Close button is below the switches");
    check(SETTINGS_CLOSE_Y + SETTINGS_CLOSE_HEIGHT + 16 <= SETTINGS_BOX_Y + SETTINGS_BOX_HEIGHT, "Close button fits inside the panel");
    check(SETTINGS_BOX_X + 30 + estimated_text_width("Background music") + 16 <= SETTINGS_TOGGLE_X, "music label does not touch its button");
    check(estimated_text_width("Changes stay until you change them.") + 2 * 16 <= SETTINGS_BOX_WIDTH, "settings hint fits the panel");

    // Story screen text.
    check(estimated_text_width("Mia: Hi, I'm Mia, the baker who runs this shop.") + 2 * 30 <= 570, "story introduction fits the dialogue box");
    check(estimated_text_width("MIA - SHOP OWNER") + 2 * 16 <= STORY_NAME_WIDTH, "name plate fits under the portrait");
    check(STORY_NAME_Y >= 416, "name plate does not cover the portrait");
}

// True when one order has sensible values for the given mode.
bool order_is_valid(const OrderSpec &order, GameMode mode)
{
    bool ok = is_in_range(order.category, DOUGH, CAKE) &&
              is_in_range(order.tier, 1, MAX_ITEM_TIER) &&
              is_in_range(order.quantity, 1, 2) &&
              order.moveLimit > 0;

    if (mode == RECIPE_MODE)
    {
        ok = ok && is_in_range(order.secondaryCategory, DOUGH, CAKE) &&
             order.secondaryCategory != order.category &&
             is_in_range(order.secondaryTier, 1, MAX_ITEM_TIER) &&
             is_in_range(order.secondaryQuantity, 1, 2);
    }
    else
    {
        ok = ok && order.secondaryCategory == NO_ITEM && order.secondaryQuantity == 0;
    }

    return ok;
}

// Each of the three orders asks for a different food category.
bool orders_use_each_category_once(const GameState &game)
{
```

```
    int seen[4] = {0, 0, 0, 0};

    for (int i = 0; i < ORDER_COUNT; i++)
        seen[game.orders[i].category]++;

    return seen[DOUGH] == 1 && seen[TART] == 1 && seen[CAKE] == 1;
}

// Random orders are different every time, so these checks only test that
// every possible result is valid, never that two runs differ.
void test_random_orders()
{
    GameMode modes[3] = {CLASSIC_MODE, RUSH_MODE, RECIPE_MODE};
    string names[3] = {"Classic Bakery", "Rush Hour", "Recipe Challenge"};

    for (int m = 0; m < 3; m++)
    {
        bool allValid = true;
        bool eachCategoryOnce = true;
        bool oldOrdersReplaced = true;
        bool movesMatchFirstOrder = true;
        bool rushAlwaysOne = true;
        bool sawOne = false;
        bool sawTwo = false;
        bool firstOrderSeen[4] = {false, false, false, false};

        for (int attempt = 0; attempt < 200; attempt++)
        {
            GameState game = GameState();
            choose_mode(game, modes[m], 0);

            // Damage the orders, then start the mode again: it must make a fresh set.
            for (int i = 0; i < order_count_for_mode(modes[m]); i++)
            {
                game.orders[i].quantity = 99;
                game.orders[i].tier = -5;
            }
            choose_mode(game, modes[m], 0);

            for (int i = 0; i < order_count_for_mode(modes[m]); i++)
            {
                if (!order_is_valid(game.orders[i], modes[m])) allValid = false;
                if (game.orders[i].quantity == 99 || game.orders[i].tier == -5) oldOrdersReplaced =
false;
                if (modes[m] == RUSH_MODE && game.orders[i].quantity != 1) rushAlwaysOne = false;
                if (game.orders[i].quantity == 1) sawOne = true;
                if (game.orders[i].quantity == 2) sawTwo = true;
            }

            firstOrderSeen[game.orders[0].category] = true;
            if (!orders_use_each_category_once(game)) eachCategoryOnce = false;
            if (modes[m] == RECIPE_MODE && game.movesLeft != game.orders[0].moveLimit)
movesMatchFirstOrder = false;
        }

        check(allValid, names[m] + ": every random order has valid values");
```

```
        check(oldOrdersReplaced, names[m] + ": starting the mode makes a fresh set of orders");
        check(eachCategoryOnce, names[m] + ": the three orders use each food once");
        check(firstOrderSeen[DOUGH] && firstOrderSeen[TART] && firstOrderSeen[CAKE],
              names[m] + ": any food can be the first order (fair shuffle)");

        if (modes[m] == RUSH_MODE)
            check(rushAlwaysOne, "Rush Hour orders ask for one item");
        if (modes[m] == CLASSIC_MODE)
        {
            check(order_count_for_mode(CLASSIC_MODE) == 5, "Classic Bakery serves five customers");
            check(sawOne && sawTwo, "Classic Bakery orders can ask for one or two items");
        }
        if (modes[m] == RECIPE_MODE)
            check(movesMatchFirstOrder, "Recipe Challenge starts with the first order's move limit");
    }

    // Beginner mode is the guided mode, so its orders never change.
    GameState game = GameState();
    choose_mode(game, BEGINNER_MODE, 0);
    check(game.orders[0].category == DOUGH && game.orders[1].category == TART &&
game.orders[2].category == CAKE,
          "Beginner orders are always dough, tart, cake");
    check(game.orders[0].tier == 2 && game.orders[1].tier == 2 && game.orders[2].tier == 3,
          "Beginner order tiers are fixed");
}

void test_item_tiers_and_random_squares()
{
    GameState game = GameState();
    choose_mode(game, BEGINNER_MODE, 0);

    // Sixteen generator clicks must fill all sixteen squares (every empty square can be chosen).
    int created = 0;
    bool tiersOk = true;
    for (int i = 0; i < 16; i++)
    {
        if (generate_item(game, (ItemCategory)(DOUGH + i % 3))) created++;
    }
    for (int row = 0; row < 4; row++)
    {
        for (int col = 0; col < 4; col++)
        {
            if (game.board[row][col].category == NO_ITEM) created--;
            if (!is_in_range(game.board[row][col].tier, 0, MAX_ITEM_TIER)) tiersOk = false;
        }
    }
    check(created == 16, "sixteen generator clicks fill the whole board");
    check(tiersOk, "generated items stay within the tier limit");
    check(!generate_item(game, DOUGH), "a full board does not accept another item");

    // When only one square is empty, that square must be the one chosen.
    game.board[3][2].category = NO_ITEM;
    game.board[3][2].tier = 0;
    check(generate_item(game, TART) && game.board[3][2].category == TART, "the only empty square is
always chosen");

```

```
    // With two empty squares, either of them can be chosen
    // (over 100 tries both are seen; the chance of missing one is far too small to matter).
    bool sawFirstSquare = false;
    bool sawSecondSquare = false;
    for (int attempt = 0; attempt < 100; attempt++)
    {
        empty_board(game);
        for (int row = 0; row < 4; row++)
            for (int col = 0; col < 4; col++)
                place(game, row, col, DOUGH, 0);
        game.board[0][0].category = NO_ITEM;
        game.board[3][3].category = NO_ITEM;
        generate_item(game, TART);
        if (game.board[0][0].category == TART) sawFirstSquare = true;
        if (game.board[3][3].category == TART) sawSecondSquare = true;
    }
    check(sawFirstSquare && sawSecondSquare, "every empty square can be chosen, including the last
one");

    // Two foods already at the top tier cannot merge into a tier that has no picture.
    empty_board(game);
    place(game, 0, 0, DOUGH, MAX_ITEM_TIER);
    place(game, 0, 1, DOUGH, MAX_ITEM_TIER);
    select_or_move_cake(game, 0, 0, 0);
    select_or_move_cake(game, 0, 1, 0);
    check(game.board[0][0].tier == MAX_ITEM_TIER && game.board[0][1].tier == MAX_ITEM_TIER &&
          game.board[0][0].category == DOUGH && game.board[0][1].category == DOUGH,
          "top-tier foods do not merge any further");

    // Random play in every mode never creates a tier outside 0 to MAX_ITEM_TIER.
    GameMode modes[4] = {BEGINNER_MODE, CLASSIC_MODE, RUSH_MODE, RECIPE_MODE};
    bool allInRange = true;
    for (int m = 0; m < 4; m++)
    {
        choose_mode(game, modes[m], 0);
        for (int step = 0; step < 1500; step++)
        {
            if (step % 3 == 0) generate_item(game, (ItemCategory)(DOUGH + rnd(3)));
            select_or_move_cake(game, rnd(4), rnd(4), 0);
            update_game(game, 0);
            if (game.screen != PLAYING_SCREEN) choose_mode(game, modes[m], 0);

            for (int row = 0; row < 4; row++)
                for (int col = 0; col < 4; col++)
                    if (!is_in_range(game.board[row][col].tier, 0, MAX_ITEM_TIER)) allInRange = false;
        }
    }
    check(allInRange, "random play never makes a tier above MAX_ITEM_TIER");
}

void test_settings_switches()
{
    GameState game = GameState();
    start_new_game(game);
    check(game.musicOn && game.soundEffectsOn, "music and sound effects both start ON");
    check(!game.settingsOpen, "settings panel starts closed");
```

```
2996
2997        toggle_music(game);
2998        check(!game.musicOn && game.soundEffectsOn, "turning music off leaves sound effects on");
2999        toggle_sound_effects(game);
3000        check(!game.musicOn && !game.soundEffectsOn, "both can be off at the same time");
3001        toggle_music(game);
3002        check(game.musicOn && !game.soundEffectsOn, "turning music on leaves sound effects off");
3003        toggle_sound_effects(game);
3004        check(game.musicOn && game.soundEffectsOn, "both can be back on");
3005
3006        open_settings(game);
3007        check(game.settingsOpen, "open_settings shows the panel");
3008        close_settings(game);
3009        check(!game.settingsOpen, "close_settings hides the panel");
3010
3011        // Settings stay until the player changes them.
3012        toggle_music(game);   // music off, effects on
3013        show_mode_menu(game);
3014        check(!game.musicOn && game.soundEffectsOn, "settings survive going back to the menu");
3015        choose_mode(game, RUSH_MODE, 0);
3016        game.screen = GAME_OVER_SCREEN;
3017        show_mode_menu(game);
3018        choose_mode(game, RECIPE_MODE, 0);
3019        check(!game.musicOn && game.soundEffectsOn, "settings survive a whole round");
3020
3021        open_settings(game);
3022        show_mode_menu(game);
3023        check(!game.settingsOpen, "returning to the menu closes the panel");
3024    }
3025
3026    // Click on the settings button, switches and Close button the way a player would.
3027    void click(GameState &game, int x, int y)
3028    {
3029        handle_mouse_click(game, x, y, 0);
3030    }
3031
3032    void test_settings_panel_clicks()
3033    {
3034        GameState game = GameState();
3035        start_new_game(game);
3036        show_mode_menu(game);
3037
3038        click(game, SETTINGS_BUTTON_X + 5, SETTINGS_BUTTON_Y + 5);
3039        check(game.settingsOpen && game.screen == MODE_MENU, "Settings button opens the panel");
3040
3041        click(game, SETTINGS_TOGGLE_X + 10, SETTINGS_MUSIC_Y + 10);
3042        check(!game.musicOn && game.soundEffectsOn, "music switch only changes music");
3043        click(game, SETTINGS_TOGGLE_X + 10, SETTINGS_EFFECTS_Y + 10);
3044        check(!game.musicOn && !game.soundEffectsOn, "effects switch only changes sound effects");
3045        click(game, SETTINGS_TOGGLE_X + 10, SETTINGS_MUSIC_Y + 10);
3046        check(game.musicOn && !game.soundEffectsOn, "music can be switched back on by itself");
3047
3048        // While the panel is open the mode cards behind it cannot be clicked.
3049        click(game, MODE_LEFT_X + 10, MODE_TOP_Y + 10);
3050        click(game, SETTINGS_BOX_X - 5, SETTINGS_BOX_Y - 5);
3051        check(game.screen == MODE_MENU && game.settingsOpen, "clicks outside the panel do nothing");
```

```
3052
3053        click(game, SETTINGS_CLOSE_X + 10, SETTINGS_CLOSE_Y + 10);
3054        check(!game.settingsOpen && game.screen == MODE_MENU, "Close button hides the panel");
3055        check(game.musicOn && !game.soundEffectsOn, "closing the panel keeps the settings");
3056
3057        click(game, MODE_LEFT_X + 10, MODE_TOP_Y + 10);
3058        check(game.screen == PLAYING_SCREEN && game.mode == BEGINNER_MODE, "mode cards work again after
    closing");
3059        check(game.musicOn && !game.soundEffectsOn, "settings are unchanged during play");
3060    }
3061
3062    int main()
3063    {
3064        load_game_sounds(); // the game logic plays sound effects
3065
3066        test_utilities();
3067        test_layout();
3068        test_screen_flow();
3069        test_generators();
3070        test_move_and_merge();
3071        test_order_complete();
3072        test_recipe_rejects_higher_tier();
3073        test_recipe_food_collection_delay();
3074        test_beginner_has_no_high_mark();
3075        test_customer_leaving();
3076        test_final_order_wins();
3077        test_rush_hour_timer();
3078        test_recipe_move_limit();
3079        test_served_food_disappears();
3080        test_hearts_only_in_rush_hour();
3081        test_back_to_menu();
3082        test_random_orders();
3083        test_item_tiers_and_random_squares();
3084        test_settings_switches();
3085        test_settings_panel_clicks();
3086
3087        return 0;
3088    }
```